

מעבדה בתכנון ספרתי מתקדם – סמסטר ב תשפ"ד

סיכום חלק ההרצאות¹

המרכז האקדמי לב, ירושלים

תואר ראשון בהנדסת חשמל ואלקטרוניקה

מרצה: מר אורי שטרו

סיכום: אליעזר טויק

הסיכום הוא חומר עזר בלבד. החומר המחייב הוא החומר המועבר בשיעור, ובנוסף, המעבדות כולן עם פתרונותיהן.

תוכן עניינים

שבוע 1	2
ערכת BASYS-3	2
חמשת הלחצנים על הלוח	3
PMOD Ports	3
Bouncing	4
משמעות מספר הדגם של רכיב ה-FPGA	4
דוגמה של קוד בשפת XDC	5
שבוע 2	6
רמות מתחים ברכיבים לוגיים (דיגיטליים)	6
Noise margin	8
LVC MOS33	8
שבוע 3	9
בעיית זמנים שעלתה במעבדות	9
השהייה בין כניסה לציאה (Propagation delay/Input-to-output delay) והשהיית החוטים (wire delay)	9
תכנון שמבוסס על ההשהייה של הרכיבים במעגל (Delay Design)	10
תכנון שמבוסס על glitch (Glitch Design)	10
חוג סגור של לוגיקה קומבינטורית (Logical Loop design)	11
זהירות משימוש ב"tri-state"	12
הדגמת טעינת התכנון של מעבדה 1 ל-BASYS3 והצגת ביצועים בפועל (מר ערן שלום)	12
שבוע 4	15
פרקי זמן חשובים בתכנון המבוסס על Flip-Flops	15
מסלולי זמנים קריטיים	16
נוסחת ה-setup	16
נוסחת ה-hold	17
דוגמה לחישוב תדר שעון בתכנון מעשי כך שיעמוד במואות ה-setup וה-hold	18
האפשרות שיציאת FF תשפיע על כניסתו	18
החשיבות שכל FF יקבל את אות השעון ישירות	18
שבוע 5	19
הכנה לבניית שעון פיזי על לוח ה-BASYS3	19
איך פועלות דיודה ונורת LED	19
בחירת ערך הנגד עבור שמירה על הדיודה	20

¹ לקורס יש חלק נוסף – מעבדות מעשיות, והוא חלק אינטגרלי מחומר הקורס, שלא כלול במלואו בסיכום זה.

20	שימוש בטרנזיסטור בי-פולארי עבור חיבור רכיבים שצורכים זרם רב או מתח גבוה ל-PMOD
21	תצורות שונות של seven segments – אנודה/קתודה משותפת (common anode/cathode)
22	חיבור רכיב ה-seven segment שעל הלוח BASYS3 אל רכיב ה-FPGA
22	צורת ההפעלה של ארבע הספרות באמצעות חיבור מינימלי של פינים
23	תכנון היררכי (הכנה למעבדה 4)
24	שבוע 6
24	מכונות מצבים סופית (Finite State Machines = FSM)
25	מימוש ב-VHDL
26	מכונות MOORE מול מכונות MEALY
28	כללים לכתיבה נכונה של מכונת מצבים ב-VHDL באמצעות enumerated type
29	סימולציה של מכונת מצבים
29	השיטות השונות (קידודים שונים) לייצוג מצבים של מכונת מצבים באמצעות Flip-Flops
31	שבוע 7
31	תכנון סינכרוני
31	תכנון זמנים עבור אות RESET
32	מימוש clock enable ללא בעיות זמנים
33	דגשים נוספים לתכנון סינכרוני מיטבי
34	שימוש ב-clock enable במכונת מצבים
35	שבוע 8
35	הצורך בסימולציית Gate level simulation
36	הקבצים הנדרשים עבור סימולציה ברמת gate level (כולל הרחבה על SDF)
36	Global Reset/Power On Reset/Reset On Configuration (ROC)
37	סימולציה מעורבת VHDL ו-Verilog
37	הדגמת סימולציית Gate level simulation
37	תופעת ה"אקורדיון"
37	הוצאת קבצי SDF ו-nelist מ-VIVADO
38	הצגת האותות הפנימיים של ה-FPGA באמצעות Logic Analyzer (מר ערן שלום)
40	שבוע 9
40	טיפול בבעיית זמנים (setup/hold) נתונה לאחר שהתגלתה
41	בדיקה והערכה (T&E = Test and Evaluation)
41	Chip-Scope / ILA
43	Translator
44	אפשרויות מתקדמות בכתיבת testbench ב-VHDL גירסה 2008

שבוע 1

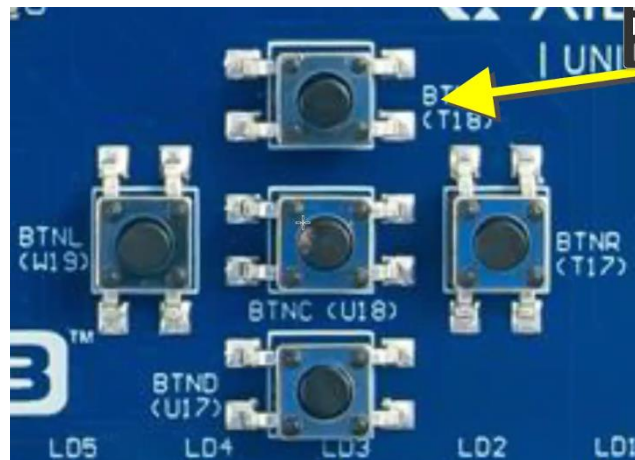
ערכת BASYS-3

בשיעור הכרנו את הלוח (board) "BASYS-3" שישמש אותנו במעבדות. הלב של ה-board הוא רכיב FPGA של חברת XILINX מהדגם XC7A35T-1CPG236C מתוך משפחת ARTIX-7. בקורס זה, נממש פיזית את התכנון ב-VHDL על גבי רכיב ה-FPGA. בנוסף לרכיב זה, ישנם על לוח BASYS-3 רכיבים נוספים המאפשרים תכנות של רכיב ה-FPGA, הכנסת INPUT לרכיב ה-FPGA המתוכנתת וקבלת OUTPUT ממנו. בנוסף, משולב בו שעון מבוסס גביש בתדר 100 MHz שמחובר בקביעות לפין W5 של רכיב ה-FPGA. באתר ה-moodle נמצאים דפי נתונים ושרטוטים, שיש בהם מידע רב על הלוח והרכיבים שעליו. הרכיב מגיע, כשמעליו ומתחתיו יש חתיכות דמויות ספוג, העשויות מפלסטיק ("חומר אנטי סטטי"). תפקידן הוא למנוע הצטברות מטענים, העלולים בהמשך להתפרק בהתפרקות אלקטרו-סטטית (ESD –

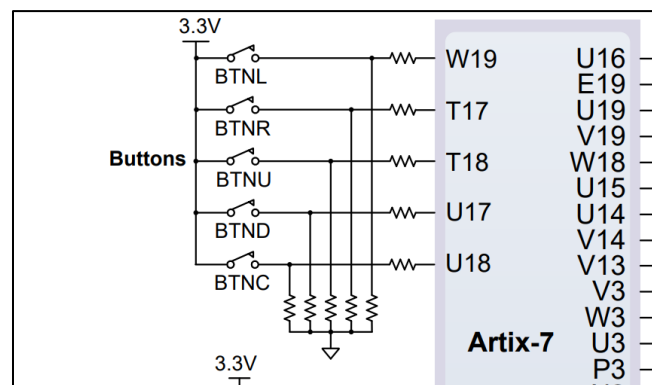
(Electro-Static Discharge), שתזיק למכשיר. למטרה זו, רכיבים אלקטרוניים רבים מגיעים בציפוי פלסטי שמשולבת בו מתכת, שמשמשת ככלוב פאראדיי, ומונעת הצטברות מטענים על הרכיב. בנוסף, גוף האדם עשוי להיות טעון ברמה כזו, שהפרש הפוטנציאלים בינו לבין הרכיב יגיע לעיתים אף ל-20,000 וולט. פריקת המטען הזה לא מזיקה לאדם (פרט להרגשת כאב רגעית, כמו שקורה לעתים כשנוגעים במתכת בימים עם לחות נמוכה²), משום שהזרם נמוך, אך עבור הרכיבים העדינים שנמצאים על ה-board של BASYS-3, זה עלול להיות הרסני. לכן, יש להיזהר מלגעת בידים חשופות בחלקים המוליכים שב-board.

חמשת הלחצנים על הלוח

על הלוח משולבים חמשה לחצני push-button, שדרכם אפשר להכניס אות חשמלי ל-FPGA. האות ייכנס לרכיב ויגרום לתוצאות שנתכנן בקוד ה-VHDL שנממש על ה-FPGA. ליד כל לחצן מופיע שמו ובסוגריים מופיע הפין ב-FPGA שאליו מחובר הלחצן:



השמות שלהם מורים על מיקומם היחסי, לדוגמה: BTNC פירושו button center, כי הוא נמצא באמצע, BTND פירושו button down, כי הוא נמצא למטה וכן הלאה. אופן פעולת הלחצנים:

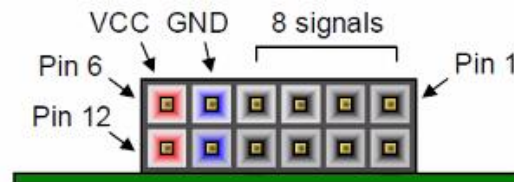


לכל פין מחובר נגד מתוך הסדרה הימנית ונגד מתוך הסדרה התחתונה. כאשר הלחצן פתוח, הפין מחובר לאדמה, והמתח בו הוא 0. אילו היה פוטנציאל אחר בפין לפני כן, הוא היה משתווה מיידית לאדמה באמצעות פריקת מטענים דרך הנגדים. נגד זה נקרא נגד pull-down. כאשר הלחצן לחוץ (סגור), הפין ב-FPGA מחובר למתח 3.3 וולט דרך הנגד הימני, ועל הנגד התחתון נופל מתח של 3.3 וולט. אם היה בפין פוטנציאל אחר לפני לחיצת הלחצן, הוא משתווה לפוטנציאל של 3.3 וולט, באמצעות פריקה דרך הנגד המחובר אליו מתוך סדרת הנגדים הימנית.

PMOD Ports

² בימים עם לחות גבוהה, המתח מתפרק לאט דרך לחות האוויר, שהיא מוליכה ברמה מסויימת, ולכן פחות מצוי שמרגישים פריקה מהירה וחדה של חשמל סטאטי.

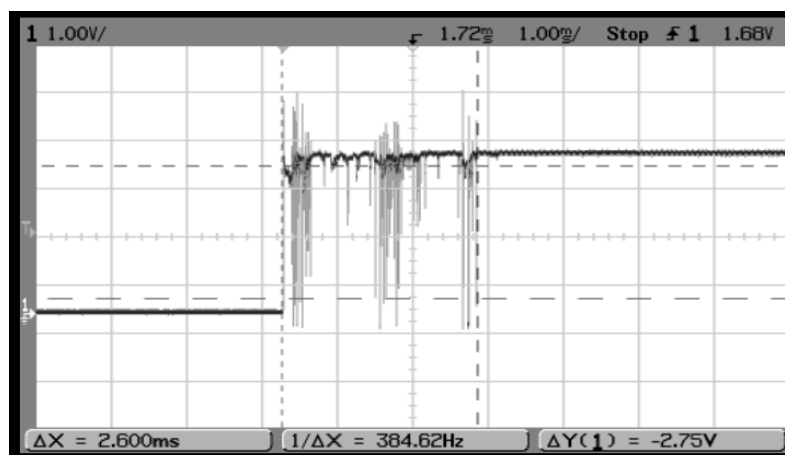
על הלוח יש ארבעה פורטים (JA,JB,JC,JXDAC) המשמשים לחיבור עם פני I/O של רכיב ה-FPGA, כשכל פורט בנוי בצורה הבאה :



פינים 6 ו-12 מחוברים למתח VCC (3.3 וולט), פינים 5 ו-11 מחוברים לאדמה, ושאר הפינים מחוברים לרגלים של ה-FPGA. בדפי הנתונים מופיע לאלו רגליים של ה-FPGA מחובר כל פין של פורטי ה-PMOD. כדי שנוכל לטפל בקלט ופלט דרך פורטי ה-PMOD, יש להגדיר בקובץ ה-xdc (שאותו נכיר בהמשך) את מיקומי האותות של המערכת ברגליים של ה-FPGA המחוברות לפיני ה-PMOD, שאנו מעוניינים לעבוד איתם.

Bouncing

כמו בכל לחצן קפיצי, גם בלחצנים שאותם הכרנו לעיל קיימת בעיית ה"bouncing". במעבר ממצב לחוץ לרפוי ולהפך, המתח אינו יציב למשך פרק זמן מסוים. בגלל ששעון המערכת מהיר מאד, פרק הזמן שבו מתח הלחצנים אינו יציב הוא משמעותי, ועלול להימשך על פני כמה וכמה מחזורי שעון. מצורף כאן גרף אמיתי³ של תופעה זו (ממערכת אחרת), שתועד באמצעות סקופ:



לבעיה זו יש פתרונות, שלא נלמד כעת. חשוב היה להכיר את הבעיה, כדי להיות מודע אליה במקרה שהיא תהיה רלוונטית.

משמעות מספר הדגם של רכיב ה-FPGA

כפי שהוזכר מקודם, דגם רכיב ה-FPGA שעל הלוח, הוא: XC7A35T-1CPG236C. כדי להבין את משמעות מספר הדגם, נעיין בדפי הנתונים של חברת XILINX. כאן מופיעה השורה הרלוונטית (המתייחסת למשפחת ARTIX-7) מתוך הקובץ 7-series-product-selection-guide.pdf שבתיקיית 'דפי נתונים, דיודות, נגדים ורכיבים' באתר ה-moodle:

ARTIX ⁷	XC	7	A	###	-1	FB	G	484	C
Xilinx Commercial	Generation	Family	Logic Cells in 1K Units	Speed Grade -1 = Slowest -L1 = Low Power -L2 = Low Power -2 = Mid -3 = Highest	Package Type CP: Wire-bond (.5 mm) CS: Wire-bond (.8 mm) FB: Bare-Die Flip-Chip (1 mm) FF: Flip-Chip (1 mm) FG: Wire-bond (1 mm) FT: Wire-bond (1 mm) SB: Bare-Die Flip-Chip (.8 mm)	V: RoHS 6/6 G: RoHS 6/6 w/Exemption 15	Nominal Package Pin Count	Temperature Grade (C, E, I)	

הסבר לפי מספר הדגם הספציפי שעל הלוח, בהתאם לשורה הזו :

³ מתוך הדף: <https://en.wikipedia.org/wiki/Switch>

XC7A – מדובר על רכיב ממשפחת ARTIX, דור 7, של חברת XILINX.

35T – ברכיב כלולים 35,000 logic cells (שעל משמעותם למדנו בסמסטר הקודם).

1- – מדובר על רכיב בעל רמת מהירות 1-, שזו המהירות הנמוכה ביותר שזמינה במשפחה הזו. עבור השימוש ברכיב כחלק מהלוח BASYS-3 זו מהירות מספיק טובה.

מכאן עוברים לפרטי האריזה:

CP – סוג האריזה (לא התייחסנו לזה בהרצאה).

G – אות שמתייחסת לתקן האירופי RoHS (לא התייחסנו לזה בהרצאה).

236 – האריזה כוללת 236 פינים.

C – העמידות בטמפרטורה היא ברמה C (commercial). זו הרמה הבסיסית, אבל היא מספיק טובה עבורנו.

דוגמה של קוד בשפת XDC

התיאום בין רגל של רכיב ה-FPGA (שמחוברת בחיבור חשמלי על הלוח ללחצן, לפין של PMOD או לכל משאב אחר שיכול לשמש להכנסה או הוצאה של אות) לבין יציאות או כניסות של התכנון ב-VHDL, נעשה בתוך קובץ xdc, שכולל גם דברים נוספים. בהמשך הסמסטר נכיר שפה זו בצורה מפורטת יותר. בהרצאה זו נראה דוגמאות בקצרה.

[דרך אגב, סינתזה בתוכנת PRECISION יוצרת לנו מעצמה קובץ XDC.]

כאן מוצג הקובץ EXAMPLE1 מתוך התיקיה 'דוגמא לאילוצי מתחים ומיקומי כניסות ב-Basys3 Board', שבאתר ה-moodle (הקוד מתייחס לתכנון ה-counter בעל 4 ביטים שהכרנו בסמסטר הקודם):

```
1
2 #####
3 # Clocks #
4 #####
5 create_clock -name CLK -period 10 -waveform {0 5} [get_ports CLK]
6
7 #####
8 # Input Delay #           I
9 #####
10 set_input_delay 0 -clock CLK -add_delay [get_ports RESET]
11
12 #####
13 # Output Delay #
14 #####
15 set_output_delay 0 -clock CLK -add_delay [get_ports {Q* TC}]
16
17
```

בשורה 5 מוגדר השעון לזמן מחזור של 10 ננו-שניות, כאשר השינוי של הערך הלוגי העלייה מתרחשת בזמן 0 והירידה בזמן 5 ננו-שניות, מה שיוצר duty-cycle של 50 אחוז. בסוף השורה מתואמת כניסת השעון הפיזית לכניסה CLK שבתכנון ה-VHDL.

בשורה 10 מוגדרת כניסת reset בעלת השהיה (delay) בעל ערך 0, כלומר ללא השהיה.

בשורה 15 מוגדרות 2 יציאות. בגלל שמוגדרות 2 יציאות שונות, הן נמצאות בסוגריים מסולסלות נוספות בתוך הסוגריים המסולסלות שבסוף השורה. ליציאה Q נוספה כוכבית כי זו יציאה בעלת כמה ביטים.

כאן מוצג חלק מהקובץ EXAMPLE2 מתוך אותה התיקיה באתר ה-moodle (שגם הוא מתייחס לתכנון ה-counter בעל 4 ביטים שהכרנו בסמסטר הקודם):

```

20 set_property PACKAGE_PIN W5 [get_ports { CLK }];
21 set_property PACKAGE_PIN U18 [get_ports { RESET }];
22 set_property PACKAGE_PIN J1 [get_ports { Q[0] }];
23 set_property PACKAGE_PIN L2 [get_ports { Q[1] }];
24 set_property PACKAGE_PIN J2 [get_ports { Q[2] }];
25 set_property PACKAGE_PIN G2 [get_ports { Q[3] }];
26 set_property PACKAGE_PIN H1 [get_ports { TC }];
27
28 set_property IOSTANDARD LVCMOS33 [get_ports { CLK }];
29 set_property IOSTANDARD LVCMOS33 [get_ports { RESET }];
30 set_property IOSTANDARD LVCMOS33 [get_ports { Q[0] }];
31 set_property IOSTANDARD LVCMOS33 [get_ports { Q[1] }];
32 set_property IOSTANDARD LVCMOS33 [get_ports { Q[2] }];
33 set_property IOSTANDARD LVCMOS33 [get_ports { Q[3] }];
34 set_property IOSTANDARD LVCMOS33 [get_ports { TC }];

```

בשורות 20-26 מוגדרות הרגליים הפיזיות של רכיב ה-FPGA (מוקף בכתום) שיחוברו לכניסות וליציאות של התכנון (מוקף בירוק).

הכניסה RESET חוברת לרגל U18 כי היא מחוברת ללחצן האמצעי מבין החמישה שעל הלוח. הלחצן האמצעי נבחר באופן שרירותי, היה אפשר לחבר את כניסה זו ללחצן אחר או לכל כניסה אחרת. בשורות 28-34 יש פרמטרים של היציאות והכניסות, בין השאר רמות המתחים שבהם הן עובדות (במקרה שלנו, הספרות 33 מייצגות מתח של 3.3 וולט).

בשורות 20-34 היינו צריכים להתייחס לכל ביט בפני עצמו. לא היה אפשר להתייחס לסדרת ביטים בתור מערך, כמו שנעשה כשהגדרנו את היציאות והכניסות בקובץ EXAMPLE1. בקטע הבא (מתוך EXAMPLE3, שנמצא באותה תיקיה באתר ה-moodle), מוגדר עבור כל כניסה או יציאה גם את רמת המתח וגם את הרגל הפיזית של ה-FPGA באותה שורה:

```

20 set_property -dict { PACKAGE_PIN W5 IOSTANDARD LVCMOS33 } [get_ports { CLK I }];
21 set_property -dict { PACKAGE_PIN U18 IOSTANDARD LVCMOS33 } [get_ports { RESET }];
22 set_property -dict { PACKAGE_PIN J1 IOSTANDARD LVCMOS33 } [get_ports { Q[0] }];
23 set_property -dict { PACKAGE_PIN L2 IOSTANDARD LVCMOS33 } [get_ports { Q[1] }];
24 set_property -dict { PACKAGE_PIN J2 IOSTANDARD LVCMOS33 } [get_ports { Q[2] }];
25 set_property -dict { PACKAGE_PIN G2 IOSTANDARD LVCMOS33 } [get_ports { Q[3] }];
26 set_property -dict { PACKAGE_PIN H1 IOSTANDARD LVCMOS33 } [get_ports { TC }];

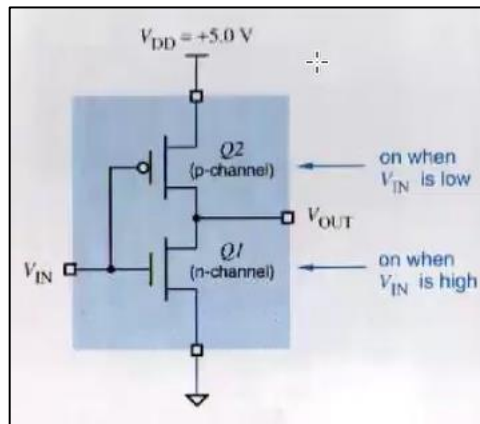
```

דרך זו תקנית כמו הדרך הקודמת, והמתכנן יבחר את הדרך שנוחה לו. בסוף ההרצאה הודגם איך לבצע implementation (place & route) באמצעות תוכנת VIVADO (אפשר גם לבצע סימולציה וסינתזה בתוכנה זו – בעת הצורך). ההדגמה לא מופיעה כאן בסיכום, כי היא פחות מתאימה לאופי של סיכום.

שבוע 2

רמות מתחים ברכיבים לוגיים (דיגיטליים)

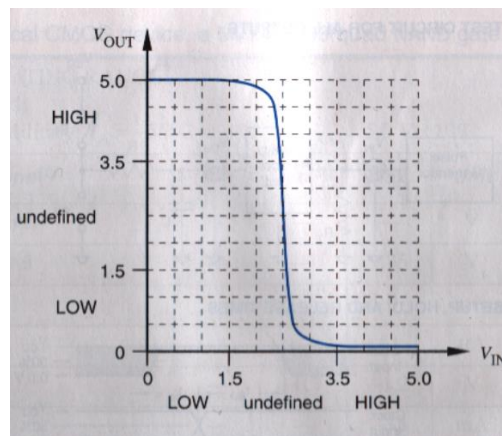
רעיון הפעולה של חומרה דיגיטלית מבוסס על ייצוג הערכים הלוגיים 0 ו-1 באמצעות שתי רמות מתחים שונות, כמו שלמדנו בסמסטר הקודם. בתור דוגמה לתזכורת, נראה את מעגל המהפך (converter) שפגשו בסמסטר הקודם, במעגל שבו המתח העליון (VDD) הוא 5 וולט:



כאשר בכניסה יש 1 לוגי, המיוצג באמצעות המתח הגבוה של המעגל, הטרנזיסטור העליון (Q_2), שהוא מסוג p-channel, סגור וכך היציאה מנותקת מהקו של V_{DD} . לעומתו, הטרנזיסטור התחתון (Q_1), שהוא מסוג n-channel, פתוח, ומחבר את היציאה ל-ground (0 וולט). כאשר בכניסה מופיע 0 לוגי בצורת מתח 0 וולט, קורה כל התהליך הפוך, כך שביציעה יש מתח של V_{DD} שמייצג ערך לוגי 1.

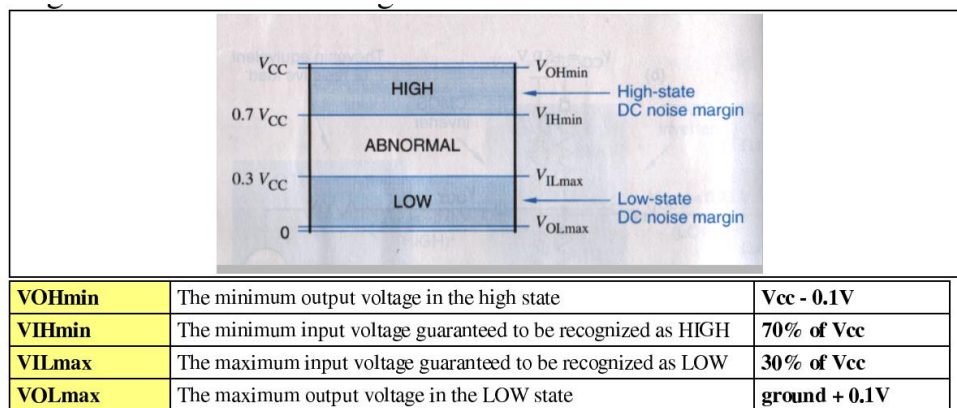
כרגיל ישנו threshold לערכי המתחים הללו ביציאה מהרכיב, וערכו הוא כרגיל כ-5% מערך ההפרש שבין הרמה הגבוהה לנמוכה. לדוגמה: אם רמת המתח המייצגת 0 היא 0 וולט ורמת המתח המייצגת 1 היא 5 וולט. יהיה threshold של 0.25 וולט. כלומר: כאשר ביציאה מהרכיב אמור להופיע ערך לוגי 0, יופיע בו מתח שערכו בין 0 ל-0.25 וולט. כאשר ביציאה מהרכיב אמור להופיע ערך לוגי 1, יופיע בו מתח שערכו בין 4.75 ל-5 וולט.

בגרף הבא, המתאר את V_{out} כפונקציה של V_{in} במהפך שראינו לעיל, אפשר לראות איך שעבור ערכי V_{in} שאינם בקצוות של 0 או 5 וולט, אבל גם אינם באמצע הלא-מוגדר, מתח היציאה V_{out} לא יהיה המתח האידאלי של 5 או 0 וולט, אלא ייכנס לתוך גבולות ה-threshold:



בגרף זה אפשר לראות, שערכי היציאה יהיו בתחום שלא חורג מגבולות ה-threshold עבור רוב הספקטרום של מתחי כניסה שונים. כלומר, גם כשמתח הכניסה רחוק יחסית מהמתח האידאלי שאמור לייצג ערך לוגי מסוים, המתח ביציאה, יהיה קרוב יחסית לערך המתח שאמור לייצג את הערך הלוגי שהיא אמורה לבטא לפי הנוסחה הלוגית של הרכיב.

עובדה זו מתבטאת גם בתרשים הבא, שנתון על ידי יצרן מסוים. שימו לב לטווח הקטן של החריגה במתחי היציאה, מול הטווח הגדול של החריגה במתחי הכניסה:



[אחת הסיבות להפרש המתחים בין הרצוי למצוי, הוא התנגדות הטרנזיסטורים, שחלק מהמתח נופל עליהם ומתחסר מהערך המקורי.]

Noise margin

המתחים שמתקבלים ביציאות של הרכיבים הלוגיים, הם אלו שייכנסו לכניסות של הרכיבים שאחריהם. העובדה שלמדנו, שתחום החריגה המתקבלת ביציאות קטן מתחום החריגה ה'מותרת' בכניסות, מותיר טווח בטחון ביניהן. טווח זה נקרא בתרשים האחרון שהוצג 'noise margin' – כלומר שוליים שמהווים את הגבול המקסימלי של 'רעש' שלא יפגע בתפקוד המערכת. כל עוד הרעש החשמלי אינו גדול משוליים אלו, ורמות המתח ביציאות של כל רכיב במערכת תהיינה 'חוקיות' עבור הרכיבים שאליהן ייכנסו מתחים אלו, המערכת תתפקד בצורה תקינה. הרעש החשמלי עלול להיווצר עקב פעולות חשמליות או אלקרומגנטיות בסביבת המערכת.

ערך השוליים האלו בתחום העליון של המתחים הוא $V_{Out\ High\ min} - V_{In\ High\ min}$ וערך השוליים בתחום התחתון של המתחים הוא $V_{In\ Low\ max} - V_{Out\ Low\ max}$.

כמו שרואים בטבלה האחרונה שהוצגה, הערכים תלויים בערכו של VCC. לכן יש לקחת בחשבון את הערך המינימלי של VCC שאפשרי במכשיר (בהתאם להוראות היצרן המופיעות בדפי הנתונים), כדי לדעת מה הערך הנמוך ביותר של ה-noise margin, שאותו אסור לעבור ויש לשמור על המכשיר מפני רעש שעלול לגרום חריגה אל מעבר לשוליים הללו.

ברכיב ה-FPGA שאנו משתמשים בו, המתח הגבוה הוא 3.3 וולט ולא 5. זאת משתי סיבות:

- המגמה בתעשייה היא להגיע למתחים וזרמים נמוכים כמה שיותר, כדי לחסוך בהספק.
- הטרנזיסטורים הולכים ונעשים קטנים יותר ויותר. לכן מתח גדול עלול לפרוץ אותם בעקבות יצירת שדה חשמלי חזק ($E=V/d$).

LVC MOS33

הערכים הספציפיים של המקסימום והמינימום בכניסות וביציאות משתנים מרכיב לרכיב. לכל רכיב יש דפי נתונים, שבהם בין השאר מופיעים רמות המתחים הספציפיים שלו. כאן מופיע חלק מטבלה שמופיעה בדפי הנתונים של רכיב ה-FPGA שנמצא על לוח BASYS3:

I/O Standard	V _{IL}		V _{IH}		V _{OL}	V _{OH}	I _{OL}	I _{OH}
	V, Min	V, Max	V, Min	V, Max	V, Max	V, Min	mA, Max	mA, Min
HSTL_I	-0.300	V _{REF} - 0.100	V _{REF} + 0.100	V _{CCO} + 0.300	0.400	V _{CCO} - 0.400	8.00	-8.00
HSTL_I_18	-0.300	V _{REF} - 0.100	V _{REF} + 0.100	V _{CCO} + 0.300	0.400	V _{CCO} - 0.400	8.00	-8.00
HSTL_II	-0.300	V _{REF} - 0.100	V _{REF} + 0.100	V _{CCO} + 0.300	0.400	V _{CCO} - 0.400	16.00	-16.00
HSTL_II_18	-0.300	V _{REF} - 0.100	V _{REF} + 0.100	V _{CCO} + 0.300	0.400	V _{CCO} - 0.400	16.00	-16.00
HSUL_12	-0.300	V _{REF} - 0.130	V _{REF} + 0.130	V _{CCO} + 0.300	20% V _{CCO}	80% V _{CCO}	0.10	-0.10
LVC MOS12	-0.300	35% V _{CCO}	65% V _{CCO}	V _{CCO} + 0.300	0.400	V _{CCO} - 0.400	Note 3	Note 3
LVC MOS15	-0.300	35% V _{CCO}	65% V _{CCO}	V _{CCO} + 0.300	25% V _{CCO}	75% V _{CCO}	Note 4	Note 4
LVC MOS18	-0.300	35% V _{CCO}	65% V _{CCO}	V _{CCO} + 0.300	0.450	V _{CCO} - 0.450	Note 5	Note 5
LVC MOS25	-0.300	0.7	1.700	V _{CCO} + 0.300	0.400	V _{CCO} - 0.400	Note 4	Note 4
LVC MOS33	-0.300	0.8	2.000	3.450	0.400	V _{CCO} - 0.400	Note 4	Note 4
LVTTL	-0.300	0.8	2.000	3.450	0.400	2.400	Note 5	Note 5

כמו שרואים בטבלה, רמות המתחים (וממילא גם ה-noise margin) תלויות בסטנדרט הספציפי שבו משתמשים במימוש התכנון באמצעות רכיב ה-FPGA. בטור השמאלי מופיעים שמות של סטנדרטים שונים של יציאות וכניסות, ובשאר הטורים את המתחים הרלוונטיים לסטנדרטים השונים. את הסטנדרט שאיתו מעוניינים לממש את התכנון קובעים בקובץ XDC. לדוגמה, בקטע הקוד שראינו בשבוע 1 מופיע כמה פעמים הביטוי IOSTANDARD LVCMOS33, שקובע שימוש בסטנדרט LVCMOS33 שמופיע כאן בטבלה.

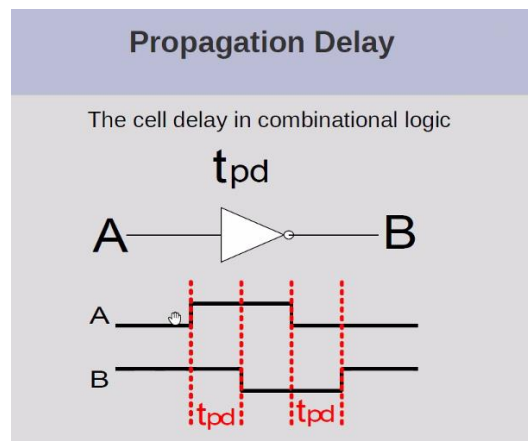
שבוע 3

בעיית זמנים שעלתה במעבדות

כאשר מבצעים implementation לתכנון של המעבדות שעשינו עד כה, תוכנת VIVADO מודיעה שיש בעיה של זמנים. כרגע אפשר 'להתעלם' מן הבעיה, אך לאורך הסמסטר (החל מהרצאה זו) נלמד לעומק את נושא הזמנים.

השהייה בין כניסה ליציאה (Propagation delay/Input-to-output delay) והשהיית החוטים (wire delay)

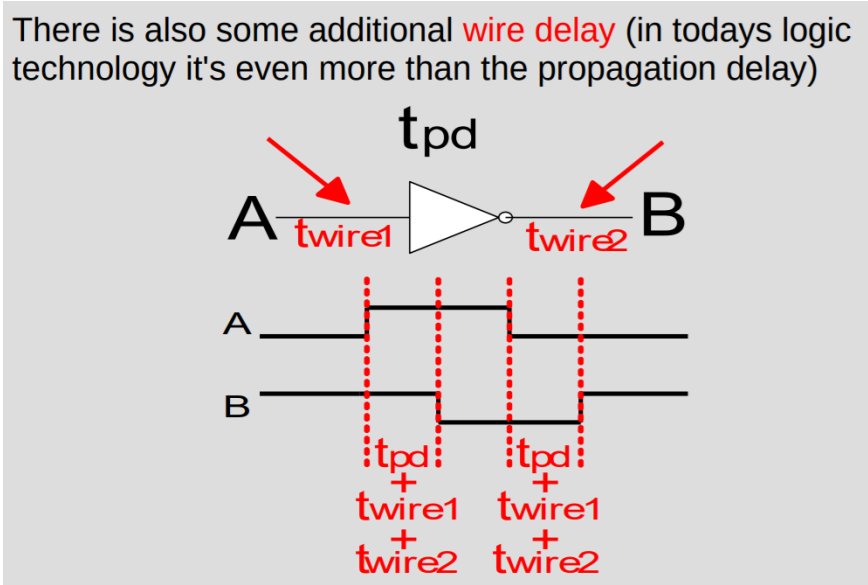
ניכנס לסוגיית מסלולי הזמנים במעגלים השונים, ונתחיל עם ההשהיות הבסיסיות במעגל. לרכיב לוגי שהגיעו לכניסותיו ערכים לוגיים מסוימים (המבוטאים ברמות מתחים) לוקח זמן להגיב, ולתת ביציאה את הערך הרצוי (בצורה יציבה) בהתאם לערכים שנכנסו לכניסותיו. כלומר מתח היציאה לא יהיה מיד המתח המצופה בהתאם למתחים בכניסות, אלא רק לאחר השהיה. ההשהיה נובעת מכך ששינוי מצב פיזיקלי של הטרנזיסטורים בתוך הרכיב לעולם לא יקרה ב-0 זמן, כל שינוי פיזיקלי אורך זמן מה. ההשהיה הזו מסומנת: t_{pd} (כלומר time of propagation delay). דוגמה עבור רכיב מהפך שהכניסה שלו עולה מאפס לאחת וחזרה לאפס, והיציאה שלו מתהפכת בהתאם, אך עם השהיה:



יש לציין של- t_{pd} של כניסה ויציאה מסוימות יש כרגיל ערכי מינימום, מקסימום וטיפוסי. כמו כן, יכול להיות שעבור מעבר מערך X לערך Y יש ערך t_{pd} שונה מהערך שלו עבור מעבר הפוך. דוגמה אפשרית עבור הרכיב המהפך הנ"ל:

	Minimum	Typical	Maximum
0 to 1 transition	2.1 ns	3.0 ns	3.5 ns
1 to 0 transition	2.2 ns	2.9 ns	3.6 ns

זמני המקסימום מכונים לפעמים 'wurst case' וזמני המינימום – 'best case'. יש לציין, שלרכיב עם כמה כניסות, יכולים להיות זמני תגובה שונים עבור שינויים שונים בכל אחת מהכניסות שלו. במקרה כזה, לכל מקרה יצוין מה ה- t_{pd} עבורו, כתלות גם בכניסה הספציפית שמשתנה. השהייה נוספת שקיימת במערכת היא השהיית החוטים (wire delay) שנובעת מכך שלוקח זמן לקצוות החוט החשמלי להגיע לערכים שווים של פוטנציאל חשמלי. השהייה זו מסומנת t_{wire} . בתרשים הבא מודגמת השהיית רכיב (מהפך) והחוטים המחוברים בינו לבין שני מיקומים במעגל:



תכנון שמבוסס על השהייה של הרכיבים במעגל (Delay Design)

אין לתכנן תכנון שבנוי על ה-delay בין הרכיבים (Delay Design). זאת משום שה-delay אינו מדויק – לרכיב יש זמן מינימום, זמן מקסימום וזמן טיפוסי, והזמן בפועל שונה בין שני רכיבים מאותו סוג. הזמן עלול גם להשתנות באותו רכיב, בעקבות התיישנות הרכיב או שינוי הטמפרטורה. כמו כן, יש wire-delay של החוטים המחוברים בין הרכיבים, והוא קשה למדידה ולשליטה. הבעיה מתעצמת כאשר מדובר בתכנון עם רכיבים רבים, שקשה מאד לסנכרן בין ההתנהגויות של כולם, ואורכי החוטים המקשרים ביניהם. בהמשך נראה דוגמאות עקרוניות לאפשרות הזו.

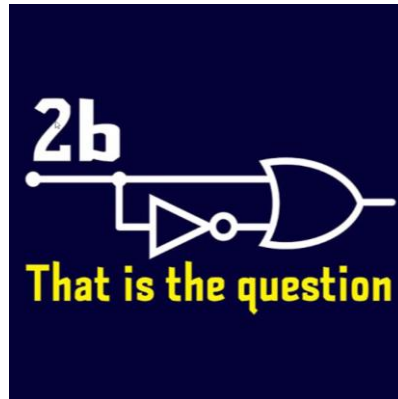
תכנון שמבוסס על glitch (Glitch Design)

Glitch היא קפיצת ערך לוגי מסוים לערך אחר (שאינו רצוי כרגיל), למשך פרק זמן קצר (ראה סיכום של קורס קודם, שבוע 3).

אין לתכנן תכנון שבנוי על ה-glitch ביציאה של רכיב (Glitch Design). ראשית, זהו למעשה סעיף של הכלל הקודם, האוסר תכנון המבוסס על delay, כי glitch נובע מ-delay. אך יש לכלל זה סיבות ייחודיות נוספות משלו:

- יכול להיות שכלי הסינתזה יעשה אופטימיזציה, ויבטל את המבנה המוצע.

- glitch יכול להיעלם עקב קיבוליות במעגל. כידוע, קיבול במעגל יכול להפוך את המעגל למסנן מעביר נמוכים, ו-glitch הוא אות בעל תדר מאד גבוה, שעלול להיות 'מסונן'.
 - אין לדעת מהו האורך המדויק של ה-glitch.
- דוגמה ל-Glitch Design היא "מעגל המלט" (Hamlet Circuit), המוצג כאן (הוא אמור תיאורטית – לפי האלגברה הבוליאנית שמתאימה לחיבור שלו, להוציא תמיד 1 לוגי. אבל בפועל, בגלל ה-delay של שער ה-NOT, אילו המעגל יקבל בכניסתו אות שעון, ביציאתו של המעגל לא יהיה 1 רציף, אלא 1 בתוספת פולסים קטנים של 0):

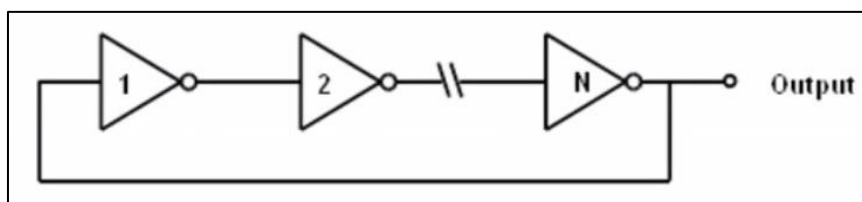


[המעגל הזה הוא רק קוריוז, ולא משמש בפועל לשום דבר ב"חיים האמיתיים". הוא בנוי על הציטוט מתוך המחזה "המלט" של שייקספיר: "to be or not to be"]

חוג סגור של לוגיקה קומבינטורית (Logical Loop design)

אין לבנות חוג סגור של לוגיקה קומבינטורית (Logical Loop design). כלומר מעגל שבו יציאת רכיב מסוים משפיעה על הכניסות שלו ללא הפסקה של מחזור שעון. לדוגמה: מעגל שבו יציאה של רכיב X נכנסת לכניסה של רכיב Y, ובנוסף יציאה של הרכיב Y נכנסת לכניסה של רכיב X. יש לציין שדבר זה אינו בעייתי אם על הקווים המחברים את הרכיבים יש Flip-Flops (ואז למעשה אין לוגיקה קומבינטורית, כי יש הפרדה באמצעות מחזורי שעון נפרדים – ראה סיכום של קורס שעבר, עמ' 10).

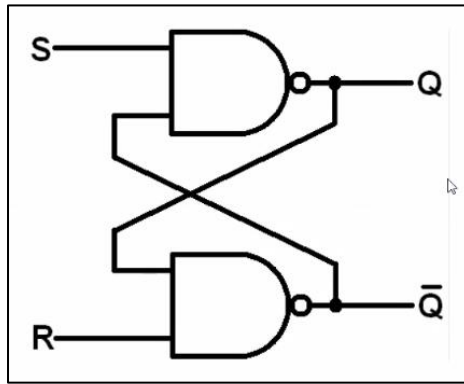
דוגמה ל-Logical Loop design היא המעגל ring oscillator, הבנוי מחוג סגור של מספר אי-זוגי של שערי NOT, כמתואר כאן:



ברמה העקרונית, המעגל הזה אמור לתת אות שדומה לאות שעון. אבל הוא מאד לא מומלץ, כי גם הוא בנוי על ה-delay, וגם הוא למעשה נחשב delay design.

בעתים נדירות משתמשים במעגל הזה, כמו עבור שעון חירום, אבל צריך "לשבת עליו שבעה נקיים" לפני שמשתמשים בו. יש לציין שהרבה פעמים כלי הסינתזה יהפוך סדרה אי-זוגית של מהפכים למהפך אחד, וסדרה זוגית לחוט.

למרות הכלל העקרוני שאוסר בניית מעגלים כאלה, יש לדעת ש-RS Flip-Flop בנוי בצורה כזו (שיש בה Logical Loop design):



ואכן, בזמן שהתכנונים היו מבוססים על Gate-Array, וה-FF היו נוצרים מחיבור של שערים לוגיים, הדבר היה מייצר בעיות. אבל ב-FPGA ה-FF מיוצרים כחלק אחד המיוצר ברמת הטרנזיסטור, תוך התחשבות בכל הבעיות העולות להיווצר.

זהירות משימוש ב"tri-state"

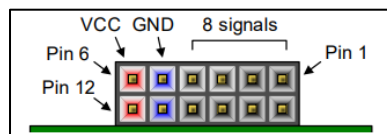
יש להימנע עד כמה שאפשר משימוש ב"tri-state", כלומר מיקום במעגל שערכו יכול להיות ערך שאינו ערך לוגי 1 או 0 (ראינו לדוגמה בסמסטר הקודם את המצב HIGH-Z). לפעמים משתמשים בתכנון כזה, בתנאי שהדבר באמת נצרך בעיקר ב-IO ring ולא ב-core logic.

הדגמת טעינת התכנון של מעבדה 1 ל-BASYS3 והצגת ביצועים בפועל (מר ערן שלום)

שלב זה הוא השלב האחרון בתהליך שמתחיל בסימולציה וממשיך לסיתזה ול-Place & Route. לאחר ש-Vivado החליטה איך למקם כל רכיב מהתכנון בתוך ה-FPGA, הגיע הזמן לממש את המעגל בפועל. הכנת הרכיב הפיזי להורדת התכנון לתוכו:

- מחברים את ה-MINUS של פרוט הסקופ ל-GROUND של הלוח דרך פין 5 או 11 של ה-PMOD. חיבור ה-ground נצרך תמיד בכל חיבור של מערכת לסקופ, כדי שתהיה נקודת ייחוס ברורה.

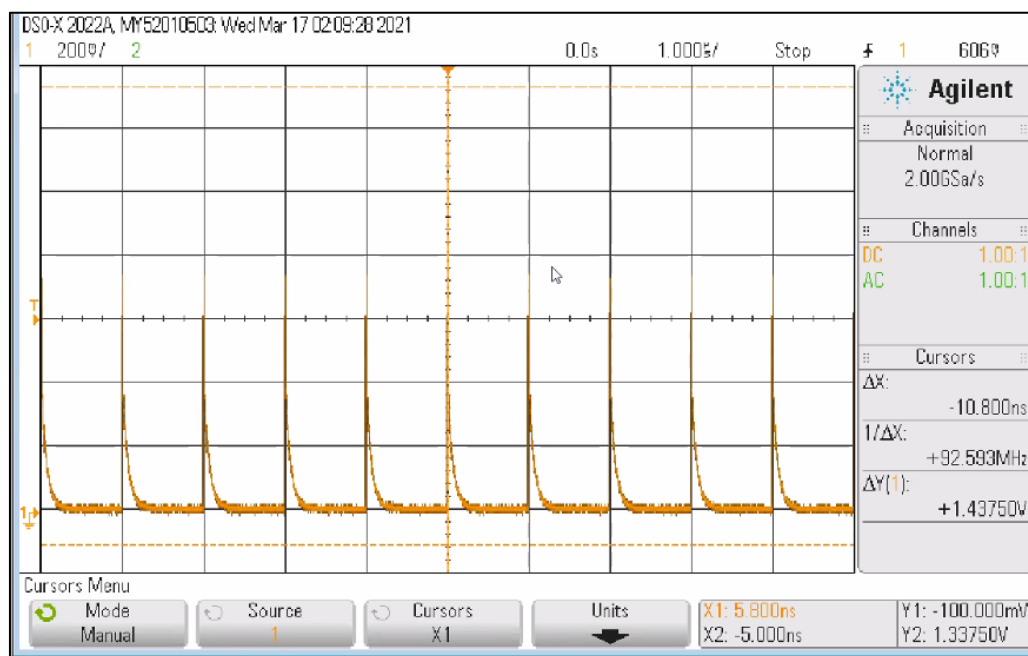
תזכורת מתוך דפי הנתונים של הלוח (הקו הירוק מסמל את לוח הפלסטיק של הרכיב):



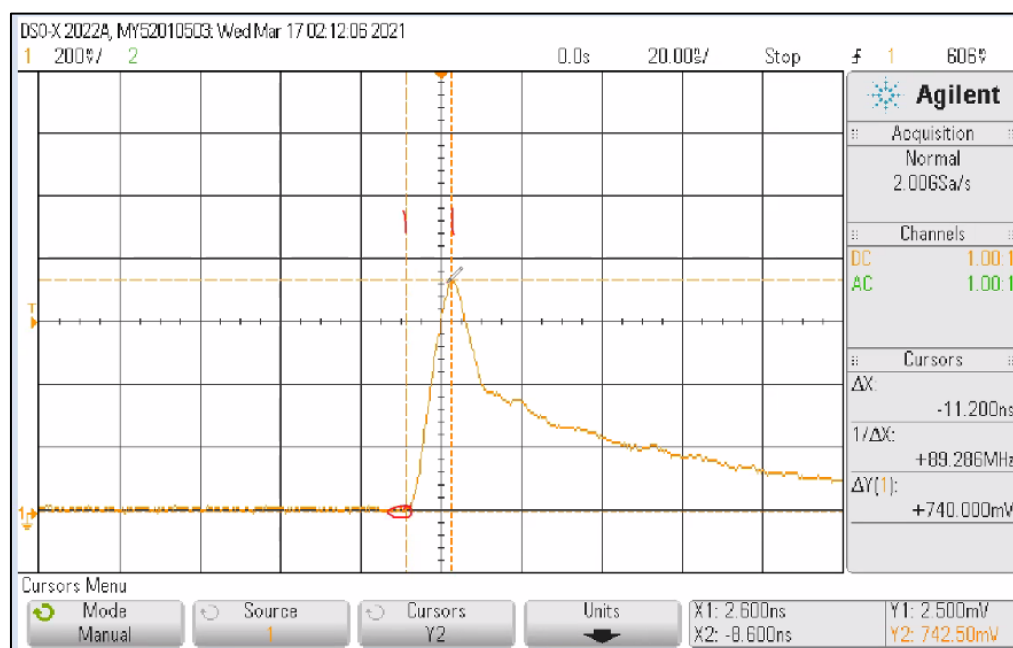
- את יציאת ה-PMOD שדרכה אנו מוציאים את האות (בחרנו את פין 1 של ה-PMOD הימני עליון (JB) שמחובר לרגל A14 של ה-FPGA), יש לחבר ל-PLUS של הסקופ. בקובץ ה-XDC יש לדאוג שאות ה-TC אכן יצא דרך פין A14 שמחובר ליציאה 1 של ה-PMOD הספציפי שבחרנו.
 - יש לחבר את כבל ה-USB בין אחת מיציאות ה-USB של המחשב, לבין הכניסה המתאימה בלוח. ב-hardware manager יש לבחור auto connect, כדי ש-Vivado תמצא את הרכיב. יש להעביר את התכנון סיתזה ו-P&R. לאחר מכן יש לייצר קובץ ביטים (bit stream), שיוכנס ל-FPGA, ויאמר לו אלו חיבורים לחבר בדיוק, ואלו רכיבים יהיו פעילים. התהליך הזה נעשה בסדר ברור, כמו שהוא מסודר בסרגל הכלים השמאלי של Vivado.
- אחרי שהורדנו את התכנון, ציפינו לראות את אות ה-TC, כפי שהכרנו אותו מהתכנון התיאורטי: בסוף כל מחזור, שאורכו מיקרו-שניה (עקב הורדת קצב שעון פי 100 שעשינו במעבדה), אות ה-TC הוא בעל ערך 1 לוגי (=3.3 וולט, לפי קובץ ה-XDC) למשך 10 ננו-שניות (אורך מחזור שעון), ואחר כך ערכו 0 לוגי (=0 וולט).

[הערה: למרות שבזמן של 10 ננו-שניות יש גם עלייה וגם ירידה של שעון הלוח, שעובד בתדר 100 מגה-הרץ, ה-TC לא משתנה בתוך פרק הזמן הזה, כי כל עוד המספר במונה (Q) לא משתנה, גם ערך ה-TC לא משתנה.]

אך זה מה שראינו בפועל (התמונות משנת תשפ"א, אבל אין שינוי מהותי):



ובהגדלה של פולס בודד:



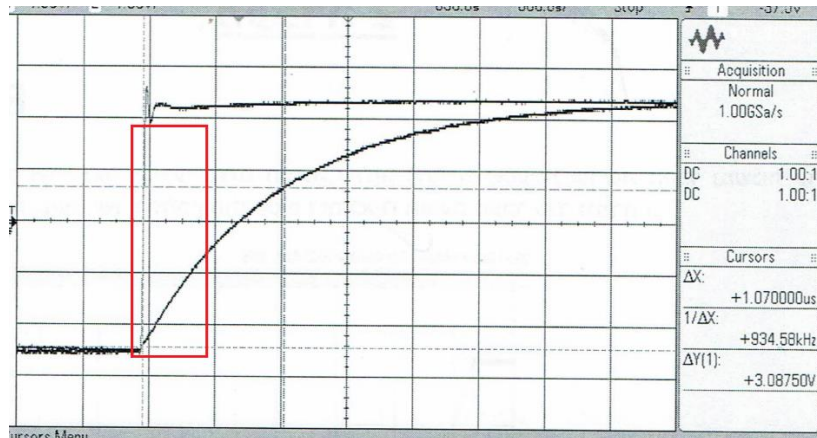
המחזוריות, כפי שציפינו, היא בתדר 1 מגה-הרץ, אבל גובה הפולס אינו 3.3 וולט, אלא בגובה שאינו מגיע לאזור של אות לוגי 1!

בנוסף, צורת הפולס מוזרה. היא לא הפולס הריבועי שציפינו לו: יש עלייה במשך כ-10 ננו-שניות, אך היא אינה מגיעה לערך שאנו מצפים לו. אחר כך מתקבלת ירידה איטית יחסית, שלא מחזירה את האות ל-0 באופן מהיר, כפי שהיינו מצפים.

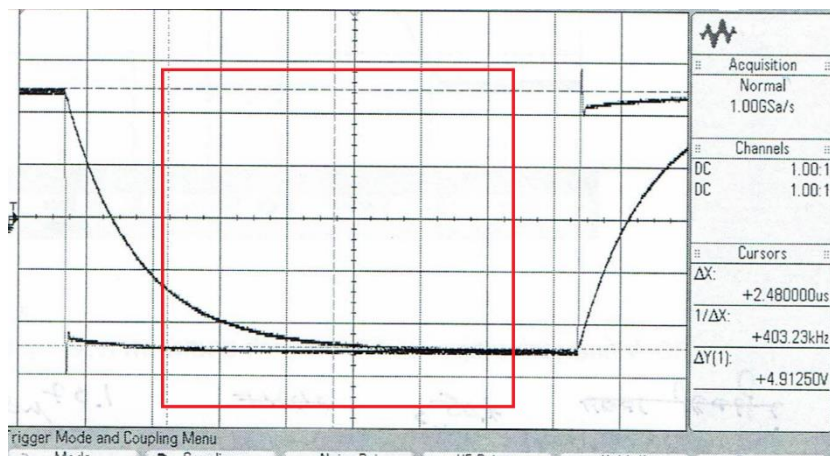
זה קורה בגלל קיבוליות (טפיליות) שיש בדרך מרכיב ה-FPGA אל הסקופ. הקיבוליות הזו נמצאת בלוח עצמו, וגם בכבל המוביל את האות. אם ניזכר בגרפי הטעינה והפריקה של קבל (יוצגו בהמשך), זה יסביר לנו גם את ההבדל בין זמן העלייה הקצר (בדומה לתחילת טעינת קבל ממצב 0, כשהזרם גבוה) לבין זמן הירידה הארוך, הדומה לפריקה מטעינה לא מלאה – כי בזמן העלייה לא היה מספיק זמן לטעינה מלאה (ונראה כמו הסוף של פריקה מלאה, כשהזרם נמוך יותר).

הרחבה של ההסבר :

כך נראית טעינת קבל (ההתחלה מודגשת – ראו את המהירות של העלייה. זה מה שקורה בעליית ה-TC):



וכך נראית פריקת קבל (הסוף מודגש – ראו כמה הירידה איטית בשלב הזה, זה מה שקורה בירידת ה-TC):



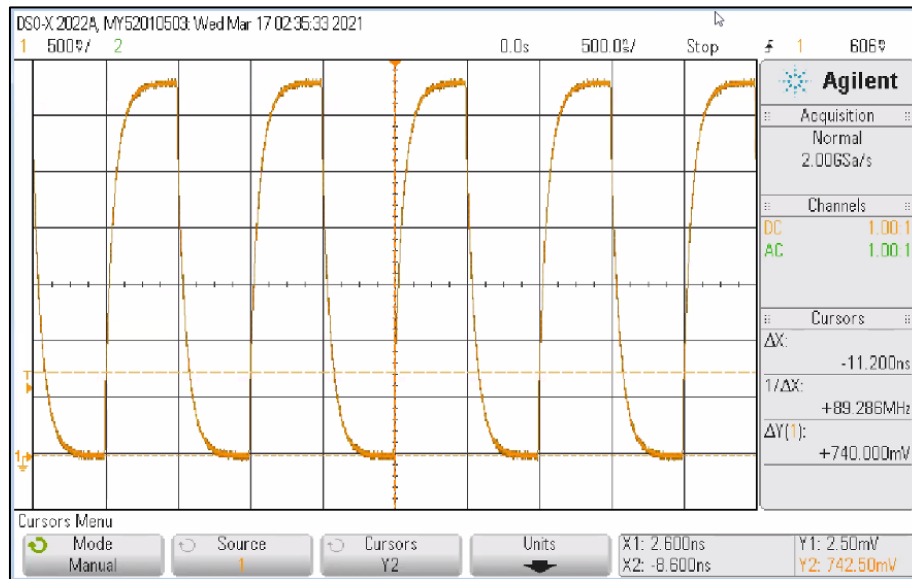
הבעיה נקראת "Rise Time" ו"Fall Time", והיא חלק מנושא גדול שנקרא "Signal Integrity".

[הערה: זה לא קשור לבעיות הזמנים ש-Vivado מתריע עליהם. כי כאן מדברים על בעיות זמנים מחוץ לרכיב ה-FPGA, בשונה מבעיות הזמנים שקיימות בתוכו.]

אם נגדיל את רוחב הפולס, יהיה מספיק זמן לטעינה מלאה ומספיק זמן לפריקה מלאה. שינינו את ה-Duty Cycle של TC ל-50 אחוז. כלומר, אורך פולס ה-1 של TC יהיה 500 ננו-שניות (=50 מחזורי שעון). בזמן זה, הפולס יצליח להגיע לגובה של 3.3 וולט, כי כל הקבלים הטפיליים ייטענו. שינינו את ע"י שינוי התנאי שמשפיע על ערך TC.

[הערה: כאשר משנים את התכנון, אי אפשר ליצור מיד קובץ ביטים. צריך לעבור שוב את הסינתזה וה-P&R.]

זו התוצאה שראינו בסקופ (גובה כל משבצת הוא 500mV, גם כאן התמונה משנת תשפ"א):

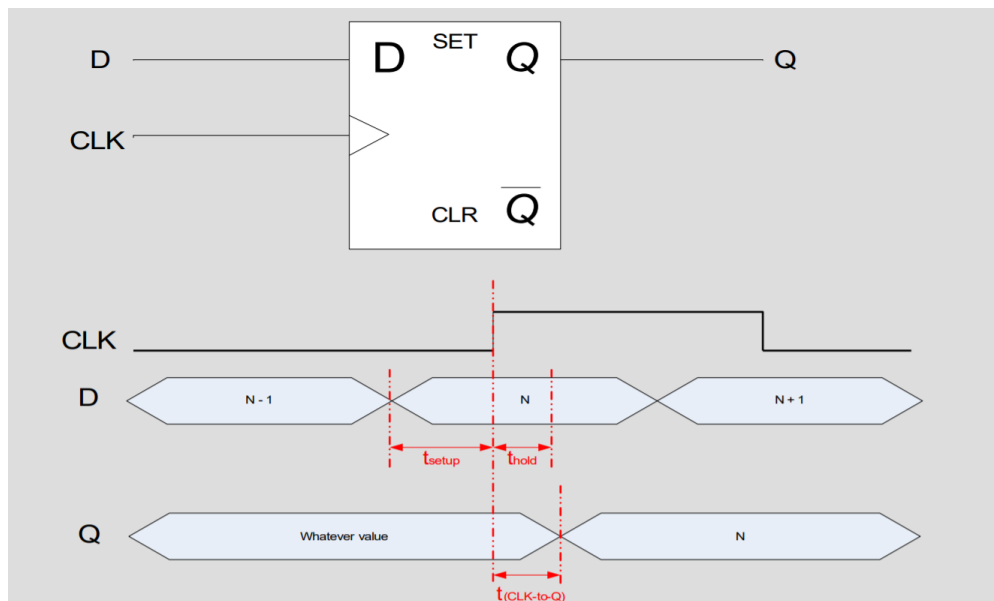


שימו לב: העלייה והירידה בעלות אותו אורך, כי מדובר על טעינה מלאה ופריקה מלאה. זאת בשונה ממה שהיה לפני כן.
כדי לתקן את הקיבוליות, לפחות באופן חלקי, יש אפשרות להשתמש בפרוב שאפשר לכייל את הקיבוליות שלו.

שבוע 4

פרקי זמן חשובים בתכנון המבוסס על Flip-Flops

בשקף הבא מוצג Flip-Flop מסוג D, עם גרף זמנים, שבו ישנם שלשה פרקי זמן חשובים:



הסבר על שלושת הזמנים הללו ומשמעותם:

כדי שהעברת הערך מ-D ל-Q תיעשה בצורה תקינה, צריך להקפיד על 2 תנאים:

- הערך שאמור לעבור ל-Q יהיה יציב ב-D פרק זמן מסוים לפני אות השעון. פרק זמן זה נקרא

t_{setup}

- הערך אמור להמשיך ולהישאר יציב ב-D במשך פרק זמן מסוים נוסף אחרי השעון. פרק זמן זה נקרא t_{hold} .

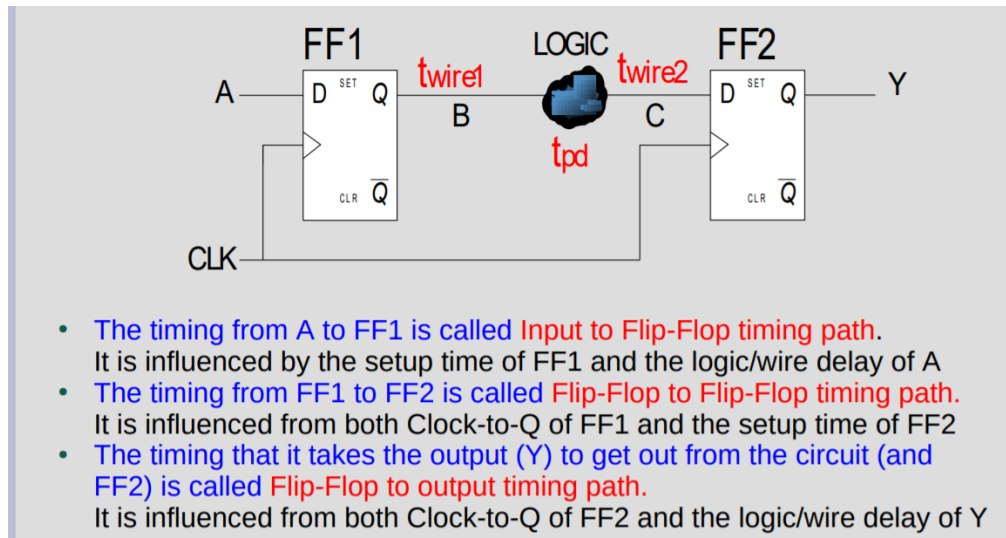
אם התנאים האלה מתקיימים, הערך יעבור בצורה תקינה. אי הקפדה על שני הזמנים הללו, עלולה להכניס את ה-FF למצב של מטא-סטביליות, שבו הערך ביציאת ה-FF לא יהיה יציב.

פרק הזמן שיעבור בין פעימת השעון לבין הצגה יציבה של הערך שהגיע מ-D בתוך Q, הוא $t_{CLK-to-Q}$.

הזמנים t_{setup} ו- t_{hold} הם תנאים להצלחה, והזמן $t_{CLK-to-Q}$ הוא תוצאה, של תכונות ה-Flip-Flop.

מסלולי זמנים קריטיים

כשמדברים על Flip-Flops מחוברים ישנם 3 מסלולי זמנים קריטיים, המושפעים מההשחיות השונות שראינו. בשקף המופיע כאן מפורטים מסלולי הזמנים הקריטיים, ומה שמשפיע על כל אחד מהם:

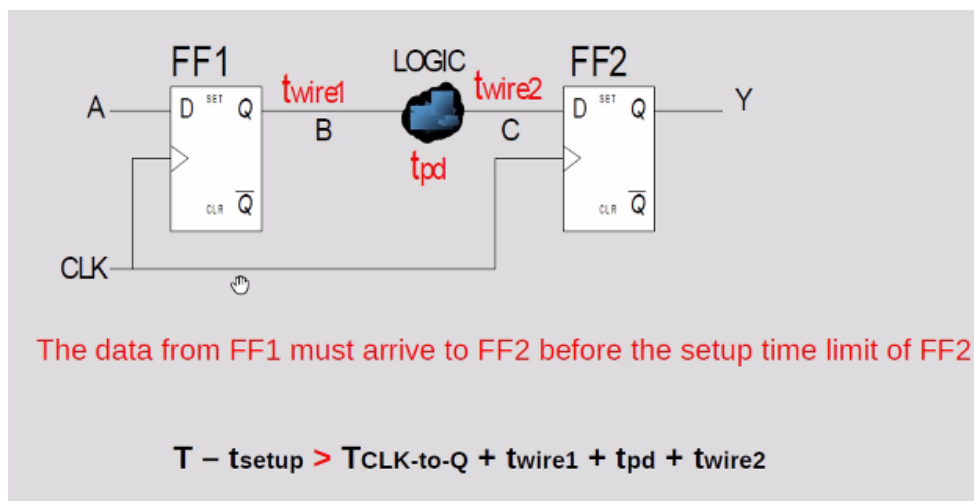


לגבי המסלול האמצעי: יש לציין, שבנוסף למה שמצוין במלל שבשקף, גם זמן ההשהייה של הקומבינטוריקה הלוגית (t_{pd}) ישפיע על הזמן של המסלול הקריטי.

הזמן שיעניין אותנו בעיקר בקורס זה הוא הזמן האמצעי, שנמצא בתוך התכנון שלנו, בשונה מהשנים הנוספים שצריכים להיות מטופלים מחוץ לתכנון.

נוסחת ה-setup

כפי שאמרנו, כדי להעביר מידע בין Flip-Flops, צריך שהמידע בכניסה ל-FF יהיה יציב למשך t_{setup} לפני עליית השעון. כדי שהתנאי יתקיים במעגל מעשי, שבו ישנו חיבור (דרך לוגיקה קומבינטורית) בין שני FF, חשוב להקפיד על "נוסחת ה-setup" או "אי-שוויון ה-setup":



T הוא זמן המחזור של שעון המערכת, הביטוי $T_{CLK-to-Q}$ מתייחס לפרק הזמן הזה עבור FF1

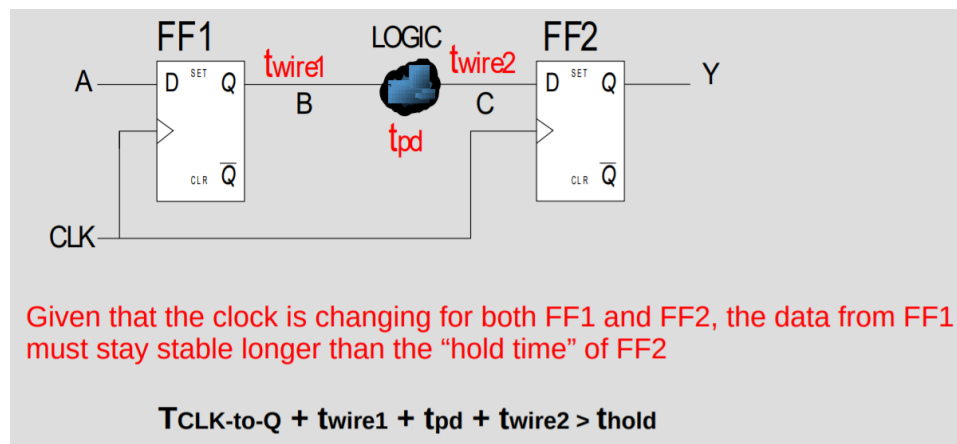
והזמן t_{setup} מתייחס לפרק הזמן הזה עבור FF2

למעשה, הנוסחה הזו היא הגיון פשוט: צריך להקפיד שהמידע מ-FF1 יגיע ל-D של FF2, כך שהוא יהיה שם מספיק זמן לפני מחזור השעון, כדי שהמידע יעבור בין D ל-Q בתוך FF2 בצורה תקינה. הזמן שהצד השמאלי של המשוואה עודף על הצד הימני נקרא slack, ואם הצלחנו להגיע למצב שיש עודף, אנו מציינים זאת "slack met". אם העודף הוא שלילי, כלומר המשוואה לא התקיימה, אנו מציינים "slack violated".

אם המשוואה לא מתקיימת בכל המסלולים שבין FF ל-FF הקיימים בתכנון (עם דגש על המסלול הארוך ביותר), יש לשנות מיקומים או הרכבים במעגל או לשנות את תדר השעון. אפשר לומר שהתכנון הספציפי ב-VHDL מכתוב קצב מקסימלי של השעון, שאותו אין לעבור (כדי שזמן המחזור T לא יהיה קצר מדי). אם כן, תדר השעון של המערכת חייב לאפשר קיום של נוסחת ה-setup. אך יש לציין נקודה חשובה בנושא קביעת תדר השעון הלכה למעשה: כשמתכננים בפועל את השעון, לא לוקחים את התדר המדויק שמאפשר את קיום הנוסחה, אלא מקובל לקחת טווח ביטחון באזור של 10 אחוזים. זאת בגלל אי-דיוקים שעלולים להיות בגביש שיוצר את אות השעון, שינוי התנהגות של רכיבים בעקבות שינוי טמפרטורה ודברים נוספים שעלולים להשתנות.

נוסחת ה-hold

בנוסף להקפדה על נוסחת ה-setup, כדי שהערך ייטען לתוך ה-FF בצורה מיטבית, יש להקפיד שהערך הלוגי יישאר בכניסת ה-FF מספיק זמן אחרי עליית השעון. לשם קיום תנאי זה, יש את נוסחת ה-hold. בכדי לדאוג לקיום t_{hold} מספיק, יש לקיים נוסחה זו בנוסף לנוסחת ה-setup. נוסחת ה-hold מוצגת כאן:



הביטוי $T_{CLK-to-Q}$ מתייחס לפרק הזמן הזה עבור FF1 והזמן t_{hold} מתייחס לפרק הזמן הזה עבור FF2.

נוסחה זו אינה תלויה בזמן המחזור של השעון.

גם כאן, הזמן שהצד השמאלי של המשוואה עודף על הצד הימני נקרא slack, ואם הצלחנו להגיע למצב שיש עודף, אנו מציינים זאת "slack met". אם העודף הוא שלילי, כלומר המשוואה לא התקיימה, אנו מציינים "slack violated".

אם יש בעיה בקיום משוואה זו, אפשר שכלי ה-Place & Route יוסיף השהייה בין הרכיבים, כך שלא יגיע מידע חדש לכניסת ה-FF לפני שהמידע הקודם נטען בהצלחה. אמנם למדנו שאסור להשתמש ב-delay design, אך במקרה זה אין חשש, כי הכלים האוטומטיים עושים זאת, מתוך התחשבות בכל האפשרויות, ומתוך לקיחת מקדמי ביטחון, ולא המהנדס באופן ידני.

דרך אגב, יש לציין שאי קיום חריף של משוואת ה-setup יפגע גם ב- t_{hold} .

דוגמה לחישוב תדר שעון בתכנון מעשי כך שיעמוד במואות ה-setup וה-hold

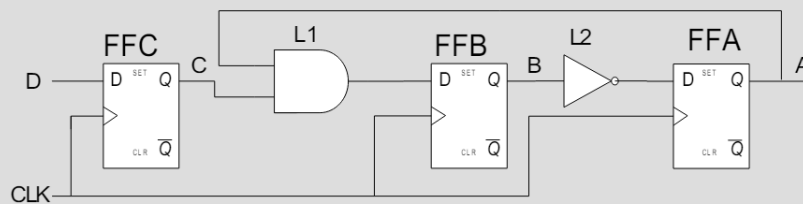
החישוב עבור נוסחת ה-setup צריך להתמקד במסלול FF to FF הארוך ביותר (= זה שיש בו את ההשהיה המירבית), כי בו יש את הסיכון הגבוה ביותר:

Design Example

Determine the maximum frequency that a given logic circuit can work at:

```
process
begin
  wait until rising_edge(CLK);
  A <= not(B);
  B <= C and A;
  C <= D;
end process;
```

AND delay	3.0 ns
INV delay	1.0 ns
Wire delay	0 ns
Setup time	1.5 ns
Hold time	0 ns
Clock to Q	1.2 ns



There are several Flip-Flop to Flip-Flop timing paths (critical paths) but the longest one of them is the path from FFA to FFB via the AND gate.

$$T - t_{\text{setup}} > T_{\text{CLK-to-Q}} + t_{\text{wire1}} + t_{\text{pd}} + t_{\text{wire2}}$$

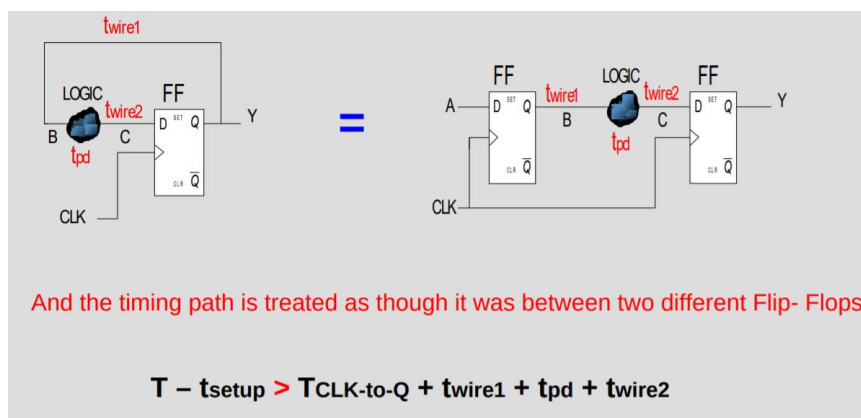
$$T - 1.5\text{ns} > 1.2\text{ns} + 0 + 3\text{ns} + 0$$

$$T > 1.5\text{ns} + 1.2\text{ns} + 3\text{ns} \implies T > 5.7\text{ns} \implies F < 175\text{MHz}$$

בדוגמה הספציפית נתון ש: $t_{\text{hold}} = 0$ כך שאין צורך לחשב את משוואת ה-hold. אם הוא היה גדול יותר, היה צריך לדאוג לכך שתדר השעון יעמוד גם במשוואה זו, ולהתמקד במסלול הקצר ביותר (= זה שיש בו את ההשהיה הקטנה ביותר), כי בו יש את הסיכון הגבוה ביותר מבחינת נוסחת ה-hold.

האפשרות שיציאת FF תשפיע על כניסתו

ראינו לעיל, שלא טוב שיציאת רכיב תשפיע על הכניסה שלו (Logical Loop design), אך אם הרכיב הוא FF, אין בעיה. בתרשים הבא נראה שיש אפשרות להתייחס למדידות הזמנים במקרה של FF שיציאתו משפיעה על כניסתו, כאילו מדובר בשני FF שונים:



כפי שראינו, תדר השעון של המערכת חייב לאפשר קיום של נוסחאות ה-setup וה-hold. תכנון שלא יעמוד בנוסחת ה-hold ייחשב כעובר על "עברת hold", ותכנון שלא יעמוד בנוסחת ה-setup ייחשב כעובר על "עברת setup".

החשיבות שכל FF יקבל את אות השעון ישירות

כל FF בתכנון צריך לקבל לתוך כניסת ה-clock דווקא את אות השעות המקורי, ולא כל מיני אותות שמושפעים מהשעון, אך עוברים דרך רכיבים נוספים. גם אם בתאוריה האותות האלו הם באותו קצב של השעון, בפועל, בגלל ההשהיה של הרכיבים, הם יקבלו את אות השעון באיחור, ולא יהיה אפשר לדאוג לקיום נוסחאות ה-setup וה-hold.

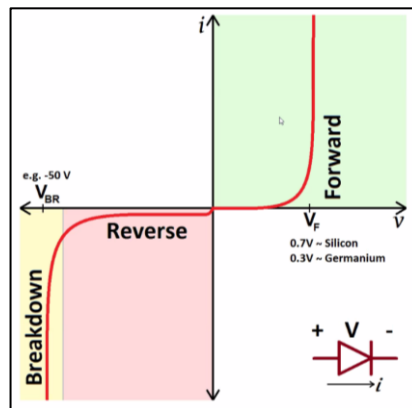
שבוע 5

הכנה לבניית שעון פיזי על לוח ה-BASYS3

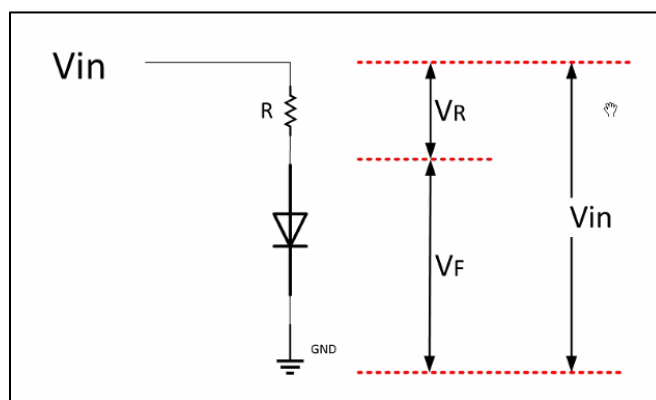
לקראת מימוש התכנון של השעון על לוח BASYS3, כולל שימוש בתצוגת ספרות בצג seven-segment (כמו שיש בשעוני יד דיגיטליים) נלמד כמה עקרונות חשובים בנושא נורות LED, הפעלת רכיבים באמצעות FPGA ותצוגת seven segment.

איך פועלות דיודה ונורת LED

נורת LED היא דיודה שפולטת אור. היחס בין הזרם למתח בדיודה מוצג בגרף הבא:



בצד ימין של הגרף, מתוארת התנהגות הדיודה ב'מתח קדמי'. עד לרמת מתח מסוימת, המסומנת בסימון V_F (מתח קדמי), לא יזרום בה זרם, או לפחות כמעט לא יזרום בה זרם. כשנפול עליה מתח V_F , אפשר להזרים בה כל זרם שנרצה, כמעט בלי להגביר את המתח מעבר ל- V_F . בצד שמאל מתוארת התנהגות הדיודה ב'מתח אחורי'. כאשר המתח גדול מספיק, הדיודה 'נפרצת', וזרם בה זרם הפוך מהכיוון ה'תקני' שלה – מהאנודה (ה'פלוס') לקתודה (ה'מינוס'). נורת לד (דיודה פולטת אור – Light Emitting Diode) היא דיודה, הפועלת גם היא בצורה דומה למה שראינו, אך בנוסף יש לה תכונה של פליטת אור כאשר זרם בה זרם. כדי שהאור יהיה בעל עוצמה שתראה לעין, יש צורך בזרם בעל ערך מינימלי מסוים. בעקבות החומרים הספציפיים שמהם עשויות דיודות ה-LED, המתח הקדמי של LED, שממנו ומעלה יזרום בה זרם והיא תאיר, הוא באזור 2 וולט. זרם גבוה מדי עלול לשרוף את הדיודה, ולכן מקובל לחבר אותה לנגד בטור. כשהזרם גדל, גם המתח על הנגד גדל (לפי חוק אוהם), וכך הזרם בדיודה מוגבל לזרם שלא יפיל על הנגד מתח ש"יגנוב" מהדיודה את המתח שהיא צריכה כדי שימשיך לזרום בה זרם (V_F).



בחירת ערך הנגד עבור שמירה על הדיודה

נסתכל בתרשים שראינו זה עתה. כדי לבחור נכון את ערך הנגד (R), צריך לדעת מה ערכם של V_{in} , של V_F ושל הזרם הרצוי בדיודה (I_F). את הערכים של הזרם והמתח הרצויים בדיודה נמצא בדפי הנתונים שלה או בדפי הנתונים של המערכת שכוללת אותה. ערכו של V_{in} תלוי במקור המתח שמספק את המתח והזרם למערכת.

נבחר כדוגמה: $V_{in}=5\text{Volt}$ $V_F=2\text{Volt}$ $I_F=20\text{mA}$. עתה נחשב:

המתח על הנגד הוא: $V_R = V_{in} - V_F = 5 - 2 = 3\text{Volt}$

הזרם הרצוי בנגד הוא אותו זרם כמו הזרם הרצוי בדיודה, כי הם מחוברים בטור.

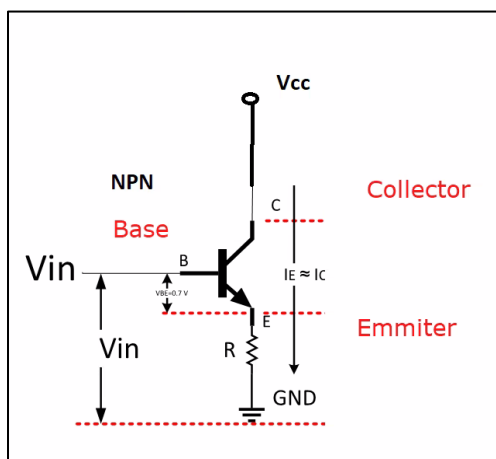
אם כן, לפי חוק אוהם מתקיים בנגד: $V_R = R \cdot I_F$ אם כן: $R = V_R / I_F = 3 / 0.02 = 150\text{ Ohm}$

יש לציין, שבפועל אי אפשר להשיג נגד בכל ערך שיהיה, כי נגדים מיוצרים בערכים מתוך סדרת ערכים אוניברסלית. אם אין נגד שהוא בדיוק בערך שמצאנו, ניקח ערך גדול מעט, כדי להיות בטוח שלא ייווצר בדיודה זרם גבוה מדי.

שימוש בטרנזיסטור בי-פולארי עבור חיבור רכיבים שצורכים זרם רב או מתח גבוה ל-PMOD

כאשר מחברים רכיבים חיצוניים שיופעלו על ידי יציאות ה-FPGA, נצטרך לפעמים לספק להם יותר זרם או יותר מתח ממה שיכול לספק רכיב ה-FPGA. במקרים אלו לא נוכל להזין מהפינים של ה-FPGA מתח וזרם ישירות לרכיב. מצד שני אנחנו רוצים לשלוט על הכיבוי וההדלקה של הרכיב באמצעות ה-FPGA. אחד הפתרונות לבעיה הזו הוא שימוש בטרנזיסטור בי-פולארי מסוג NPN כמפסק.⁴ נסביר מעט על ההתנהגות של הטרנזיסטור הזה.

חלקי הטרנזיסטור:



⁴ישנם סוגים שונים של טרנזיסטורים, ולחלק מהם יש כמה שימושים. בשלב זה אנחנו מתמקדים בטרנזיסטור בי-פולארי מסוג NPN, ורק בשימוש בו כמפסק. הנושא של טרנזיסטורים נלמד בהרחבה בקורסים אחרים.

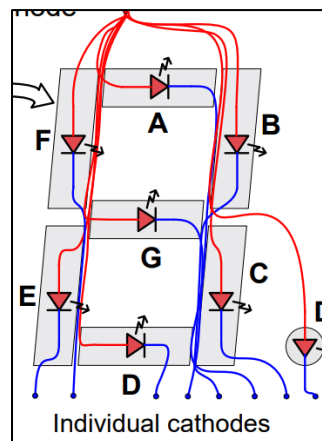
עיקר הזרם זורם בטרנזיסטור מה-collector אל ה-emitter, אך הזרם הזה נשלט על ידי המתח שבין ה-base לבין ה-emitter:

- אם המתח בין ה-base ל-emitter יהיה $0.7V$ ומעלה (זה דורש זרם קטן מאוד לתוך ה-base), הטרנזיסטור יהיה ברוויה (saturation), והזרם העיקרי – שזורם מה-collector אל ה-emitter – יזרום בלי הפרעה. במקרה זה, יידרש גם שהמתח בין ה-collector ל-emitter יהיה בערך $0.2V$. אין שליטה ישירה על המתח הזה, אבל בתכנון חלוקת המתחים בין הרכיבים צריך להשאיר 0.2 וולטים רזרביים עבור המתח הזה.
 - אם המתח בין ה-base ל-emitter יהיה $0V$, הטרנזיסטור יהיה בקיטעון ('cut-off'), ולא יהיה זרם בין ה-collector ל-emitter. עבור ערכי מתחים באמצע יהיה מצב ביניים, שאנחנו לא מעוניינים בו בשימוש הספציפי שלנו, ולכן לא נעסוק בו, אלא נדאג תמיד שיתקיים אחד מהמצבים שצינו.
- אם נחבר את מתח ה-GROUND של ספק הזרם והמתח ל-collector, נחבר את המתח מרכיב ה-FPGA ל-base ואת הרכיב שצורך הרבה זרם בין ה-collector לבין המתח העליון (V_{CC}) של ספק הזרם והמתח, נוכל לכבות ולהדליק אותו באמצעות שינויי מתחים, בלי לצרוך דרכו הרבה זרם (המקרה הימני בתרשים בעוד כמה שורות).

תצורות שונות של seven segments – אנודה/קתודה משותפת (common anode/cathode)

כשמדברים על רכיב של seven segment, המכיל למעשה 7 נורות לד בצורת קו ועוד אחת בצורת נקודה, ומשמש להצגת ספרות, צריך לשים לב לכך שיש שתי צורות שבהן הוא יכול להיות מסודר:

- אנודה משותפת (common anode): ההדקים החשמליים בעלי הערך החיובי של נורות הלד מחוברות ביניהן, ומקבלות מתח חיובי שיכול לאפשר פעולה של נורות הלד. כדי לכבות או להדליק נורה ספציפית יש לשנות את ערך המתח החשמלי בקתודה שלו, כלומר בהדק השלילי. התרשים הבא מסודר בצורה כזו:



אפשר לומר, שכל האנודות נמצאות במתח של 1 לוגי, וכדי לדאוג שנורה ספציפית תידלק, יש צורך להזין בקתודה שלה 0 לוגי, כך שיפול עליה מפל מתח בכיוון הנכון. אם יהיה בקתודה שלה 1 לוגי, לא יהיה עליה מפל מתח, כי המתח בשני ההדקים יהיה שווה.

- קתודה משותפת (common cathode): ההדקים החשמליים בעלי הערך השלילי של נורות הלד מחוברות ביניהן, ומקבלות מתח 0. כדי לכבות או להדליק נורה ספציפית יש לשנות את ערך המתח החשמלי באנודה שלו, כלומר בהדק החיובי.

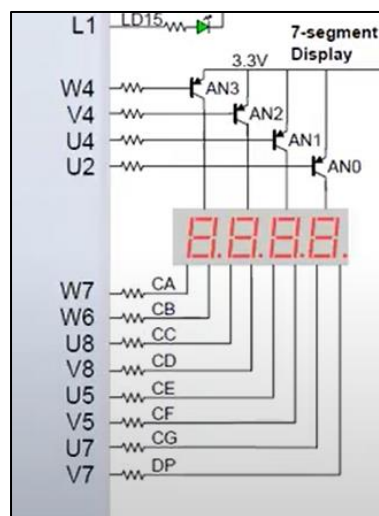
אפשר לומר, שכל הקתודות נמצאות במתח של 0 לוגי, וכדי לדאוג שנורה ספציפית תידלק, יש צורך להזין באנודה שלה 1 לוגי, כך שיפול עליה מפל מתח בכיוון הנכון. אם יהיה באנודה שלה 0 לוגי, לא יהיה עליה מפל מתח, כי המתח בשני ההדקים יהיה שווה.

בכדי ליצור שעון, נצטרך 6 ספרות. בגלל שעל לוח BASYS3 יש רק 4 ספרות, נצטרך לחבר ללוח רכיב SSD שכולל שתי ספרות נוספות דרך ה-PMOD.

בדפי הנתונים של BASYS3 מופיע, שרכיב ה-seven segment שנמצא על הלוח פועל בשיטת אנודה משותפת. לעומתו, רכיב ה-SSD שאנחנו מחברים ל-PMOD, כדי לקבל ספרות נוספות, פועל בשיטת קתודה משותפת. כל דפי הנתונים שבהם מופיע המידע הזה נמצאים במודל. צריך לשים לב לזה כשעובדים עם הרכיב הנוסף, ולכוון נכון את המתחים שיוציאו הפינים שמחוברים ל-PMOD שאליו מחובר רכיב ה-SSD.

חיבור רכיב ה-seven segment שעל הלוח BASYS3 אל רכיב ה-FPGA

בדפי הנתונים של הלוח, מופיע התרשים הבא :

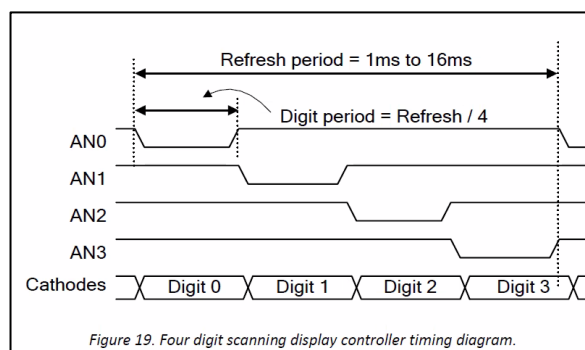


המלבן האפור-תכול משמאל מסמל את רכיב ה-FPGA. 4 היציאות העליונות מחוברות לטרנזיסטורים, כדי לשלוט על כניסת הזרם לאנודות השונות של הספרות (לכל ספרה אנודה משלה). בגלל שמדובר על טרנזיסטור PNP, דווקא הערך 0 ביציאה הוא זה שיאפשר זרם לאנודה. 8 היציאות התחתונות אחראיות לאפשר או למנוע זרם בכל אחד מ-8 נורות הLED המרכיבות את המספר (כולל זו שמייצרת את הספרה העשרונית), באמצעות שינוי מתח הקתודה. גם כאן, דווקא מתח 0 ביציאת ה-FPGA הוא זה שיפעיל את הנורה הספציפית. היציאות התחתונות אינן מחוברות דרך טרנזיסטור, כי מדובר על פחות זרם מזרם האנודה – האנודה מספקת זרם ל-8 נורות LED, והקתודה מקבלת זרם רק מנורת LED אחת. שימו לב, שיש רק 8 חיבורי קתודה, למרות שיש 4 ספרות שלכל אחת 8 קתודות, כלומר סך הכל 32 קתודות לכל מערך נורות הLED. נסביר איך אפשר להפעיל כך את כל ארבעת הספרות.

צורת ההפעלה של ארבע הספרות באמצעות חיבור מינימלי של פינים

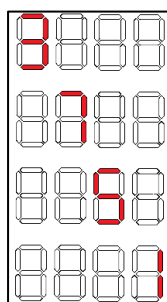
השיטה להשתמש בצורת חיבורים זו שראינו, שבה אין חיבור לכל הקתודות של הרכיב ולשלוט באמצעותה על 4 ספרות, מסתמכת על הפיזיולוגיה של העין. אם נורה נכבית ונדלקת בקצב מהיר מספיק, המח יפרש את מה שהעין כאילו הנורה דולקת כל הזמן.⁵ את התכונה הזאת אפשר לנצל לענייננו כך: מתכננים מחזור באורך של בין 1 ל-16 מילי-שניות המחולק ל-4 רבעי מחזור. בכל רבע מחזור, רק אנודה אחת תהיה פעילה (באמצעות 0 לוגי בפין המתאים, כפי שראינו לעיל). בכל רבע מחזור כזה, החיבורים של הקתודות יהיו בעלי ערכים שמותאמים לספרה שהאנודה שלה פעילה. כאן מופיע תרשים זמנים שמסביר את אופן הפעולה:

⁵ הערת הרהבה: תכונה זו מאפשרת הגברה והחלשה של האור בנורת LED באמצעות PWM.

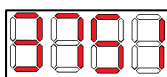


הקצב הזה מהיר מספיק כדי שהמח לא יפרש את ההדלקה והכיבוי כהבה, אלא כאילו הנורה דולקת כל הזמן באור חלש יותר.⁶

ניתן דוגמה עבור הצגת המספר 3751: העין תראה במחזוריות את 4 התמונות הבאות (מלמעלה למטה):



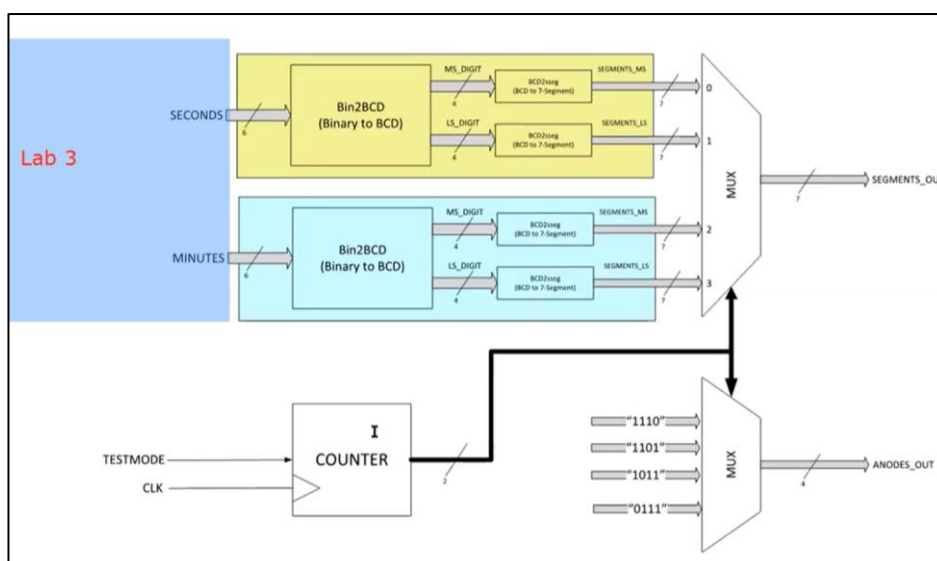
אך המח יפרש את התמונה כך:



תכנון היררכי (הכנה למעבדה 4)

במעבדה 3 יישמנו שעון שמכיל שעות, דקות ושניות. במעבדה 4 אנחנו רוצים להוציא את המספרים הללו בבסיס דצימלי, על גבי צגי seven segments. בשביל זה אין צורך להוריד חלקים ממה שכבר עשינו במעבדה 3, אלא אפשר לבנות את החלקים החדשים על גבי מה שכבר קיים, בהפרדה של התכנון לחלקים שונים בצורה היררכית.

נראה את הרעיון בצורה של תרשים בלוקים⁷:



⁶ בגלל שכל ספרה דולקת רבע מחזור, תרגיש העין כאילו הספרות בעלות עוצמה שהיא רבע מהעוצמה האמיתית שלהן. ⁷ התרשים הזה הוא תרשים עקרוני, ולא מייצג בצורה מדויקת לחלוטין את התכנון שיש לבצע בתרגיל. חשוב לתכנן בפועל מתוך חשיבה על הפרטים. בפרט חשוב לשים לב איזו ספרה מכניסים לאיזה מיקום ב-SSD.

מצד שמאל נכנסים המספרים של הדקות והשניות שאותם כבר בנינו במעבדה 3.

בתוך הבלוק Bin2BCD כל אחד מהמספרים הללו עובר קידוד מבינארי לצורה של BCD (binary coded decimal), שהיא נוחה יותר לשימוש עבור הספרות ברכיב ה-seven segment. צורת הכתיבה ב-BCD מפרידה כל ספרה עשרונית בארבע ביטים בפני עצמם. לדוגמה, המספר 25 שנכתב בבינארי: 011001 ייכתב ב-BCD: 0010,0101. הצורה הזו נוחה, בגלל שיש לנו הפרדה בין שתי הספרות העשרוניות, וזה מתאים לרכיב ה-seven segment שבו הספרות העשרוניות נפרדות.

בתוך הבלוק BCD2sseg כל אחת מהספרות עוברת תרגום מהספרה ב-BCD לקידוד שמותאם ל-seven segment, כלומר ערכי 1 ו-0 התואמים לחיבורים לקתודות של נורות הLED שאנו מעוניינים להדליק או לכבות.

בצד הימני של הבלוקים הצבעוניים, יוצאים קודים המותאמים לסדרות הקתודות של 4 הספרות של רכיב ה-seven segment. הם נכנסים ל-MUX העליון שבתרשים, כדי שבכל רבע מחזור סדרה אחת מהארבע האלה תהיה פעילה, בהתאם לספרה הפעילה באותו רבע מחזור.

ב-MUX התחתון שבתרשים, יש ארבעה קודים המותאמים כל אחד מהם להדלקה של אנודה אחת בלבד, בהתאם לספרה שאמורה להיות פעילה ברבע המחזור הספציפי.

שני ה-MUXים מקבלים אות מ-counter שדואג להחלפה במקביל של סדרות הערכים שיוצאות משניהם, בכל רבע מחזור, כך שיתאימו בכל פעם לספרה שאמורה להיות פעילה.

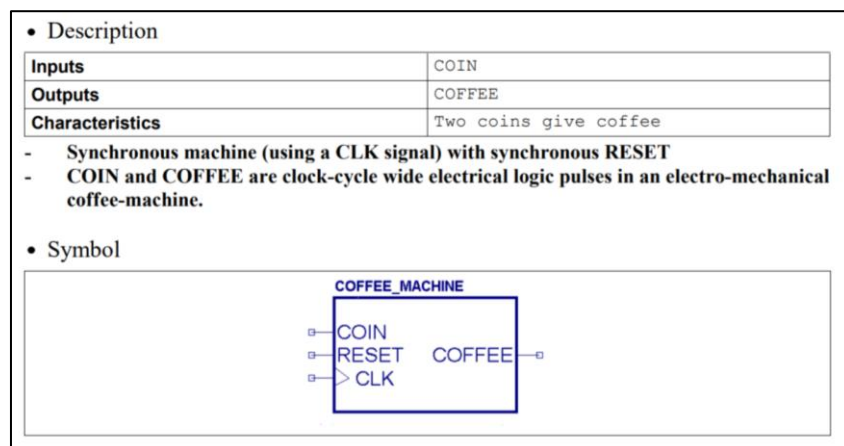
הכתיבה בפועל של תכנון היררכי נעשית על ידי כתיבת קבצים נפרדים, שבכל אחד מהם יש רכיב, וקובץ נוסף שמשתמש בכולם. את ה-syntax אפשר לראות במצגת במודל שנקראת "תיאור הירארכיה ב-VHDL".

קידודים שונים שיכולים לשמש אותנו בבלוקים השונים של התכנון ניתנים להשגה במרחבי האינטרנט. אנחנו לא הראשונים ולא האחרונים שעושים את זה.

שבוע 6

מכונת מצבים סופית (Finite State Machines = FSM)

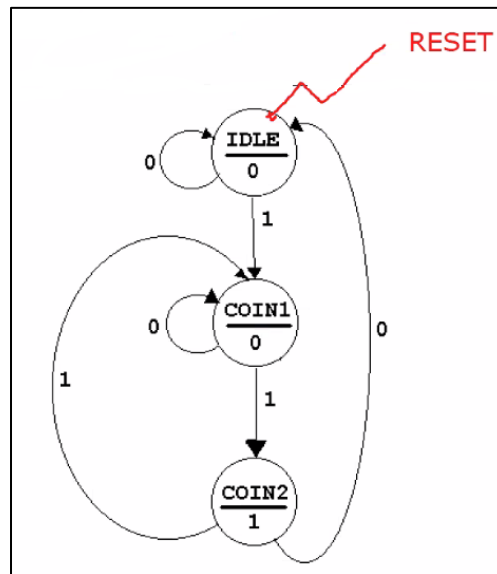
מכונת מצבים, היא טכניקה של תכנון של חומרה שעוברת בין מצבים מוגדרים. לענייננו, ההתקדמות משלב לשלב נעשית כרגיל בעליית שעון (rising edge), כלומר מדובר במכונה סינכרונית. בתור דוגמה פשוטה, שתמחיש את הרעיון, נדבר על מכונת קפה, שנותנת קפה אחר כל פעם שהיא מקבלת 2 מטבעות. נתאר אותה כמו כל מודול ספרתי שאנחנו מכירים:



המכונה יכולה לקבל INPUT שבא בעקבות הכנסת מטבע (בכל פעם מטבע בודד),⁸ ולתת OUTPUT של הוראה להכנת קפה.⁹ האותות הללו, הם ברוחב של מחזור אחד בלבד, כי אות ברוחב גדול יותר, שייקלט בכמה עליות שעון, יתפרש ככפל של ההוראה. כמו כן, כמו כל אות אחר שעובר ב-Flip-Flops, גם הם חייבים לעמוד בכל משוואות הזמנים שלמדנו.

בגלל שהמכונה סינכרונית, וממילא בהכרח כוללים Flip-Flops, חייבות להיות גם כניסות של שעון ו-RESET.

עתה נתאר את המכונה הזו בתרשים סטנדרטי ("דיאגרמת מצבים" \ "דיאגרמת בועות") של מכונת מצבים מסוג MOORE¹⁰:



ישנם 3 מצבים אפשריים, שהמכונה יכולה להיות רק באחד מהם:

- התחלה (לא התקבלו מטבעות).
- התקבל מטבע בודד.
- התקבלו 2 מטבעות.

שלושת המצבים מיוצגים בתור שלשה עיגולים, והמעברים האפשריים בין מצבים שונים מסומנים בחיצים. הספרות על החיצים מסמנים את ה-INPUT של מטבע (האם הגיע בעליית השעון הנוכחית מטבע או לא?). המספרים שבתחתית המצבים (העיגולים) מסמנים את ה-OUTPUT (האם להכין כעת קפה או לא?). אם יתקבל אות של RESET, המכונה תחזור למצב ההתחלתי – IDLE, ללא תלות במצבה הנוכחי. יש לציין: כל דיאגרמת בועות חייבת להכיל סימון המציין מצב תחזור המכונה בהינתן RESET. בנוסף, מכל מצב ממצבי המכונה צריך שיצאו חיצים למצבים נוספים (או חזרה לאותו המצב) עבור כל INPUT אפשרי, כדי שלא יהיו INPUTS שלא ברור מה המענה להן.

מימוש ב-VHDL

את המצבים האלה שראינו, ואת המעברים ביניהם, אפשר לממש ב-VHDL "ככתבם וכלשונם" באופן הבא:

⁸ בכדי לא להסתבך עם מנגנוני הבדיקה של המטבע הנכון וכי, נגדיר שה-INPUT שנקרא "COIN" הוא פולס ברוחב מחזור שעון אחד, שמתקבל ממנגנון השקילה, ומודיע שהתקבל מטבע אחד תקין.

⁹ ה-OUTPUT שנקרא "COFEE" הוא פולס ברוחב מחזור שעון אחד, שהולך לחלק במכונה שמכין בפועל קפה, ונותן לו הוראה להכין כוס אחת.

¹⁰ בהמשך נרחיב על מכונות מסוג MOORE ומכונות מסוג MEALY.

```

library IEEE;
use IEEE.std_logic_1164.all;

entity COFFEE_MACHINE is
port
(
    CLK:    in  std_logic;
    RESET:  in  std_logic;
    COIN:    in  std_logic;
    COFFEE:  out std_logic
);
end COFFEE_MACHINE;

architecture MOORE of COFFEE_MACHINE is
    type StateType is (IDLE, COIN1, COIN2);
    signal STATE: StateType;
begin
    P0: process
    begin
        wait until rising edge(CLK);
        if (RESET = '1') then
            STATE <= IDLE;
            COFFEE <= '0';
        else
            case STATE is
                when IDLE =>
                    if (COIN = '1') then
                        STATE <= COIN1;
                        COFFEE <= '0';
                    else
                        STATE <= IDLE;
                        COFFEE <= '0';
                    end if;
                when COIN1 =>
                    if (COIN = '1') then
                        STATE <= COIN2;
                        COFFEE <= '1';
                    else
                        STATE <= IDLE;
                        COFFEE <= '0';
                    end if;
                when COIN2 =>
                    if (COIN = '1') then
                        STATE <= COIN1;
                        COFFEE <= '0';
                    else
                        STATE <= IDLE;
                        COFFEE <= '0';
                    end if;
            end case;
        end if;
    end process;
end MOORE;

```

בריבוע האדום מופיע syntax שאנו פוגשים לראשונה. השורה הראשונה מגדירה משתנה חדש מסוג enumerated type (שיכול להכיל כל מיני סוגי ערכים ולא רק מספרים), שקראנו לו "StateType" (שם לא שמור, שהחלטנו לבחור), ויכול להכיל אחד משלושת הערכים הבאים: IDLE, COIN1, COIN2. השורה הזו לא יוצרת עדיין משתנה כזה, אלא רק מגדירה סוג עקרוני כזה של משתנה, ומאפשרת בהמשך ליצור משתנה ספציפי מהסוג הזה. השורה השנייה יוצרת משתנה ספציפי מהסוג "StateType", ואנו קוראים לו "STATE" (שוב, לא שם שמור). נרחיב עוד על ה-syntax בהמשך.

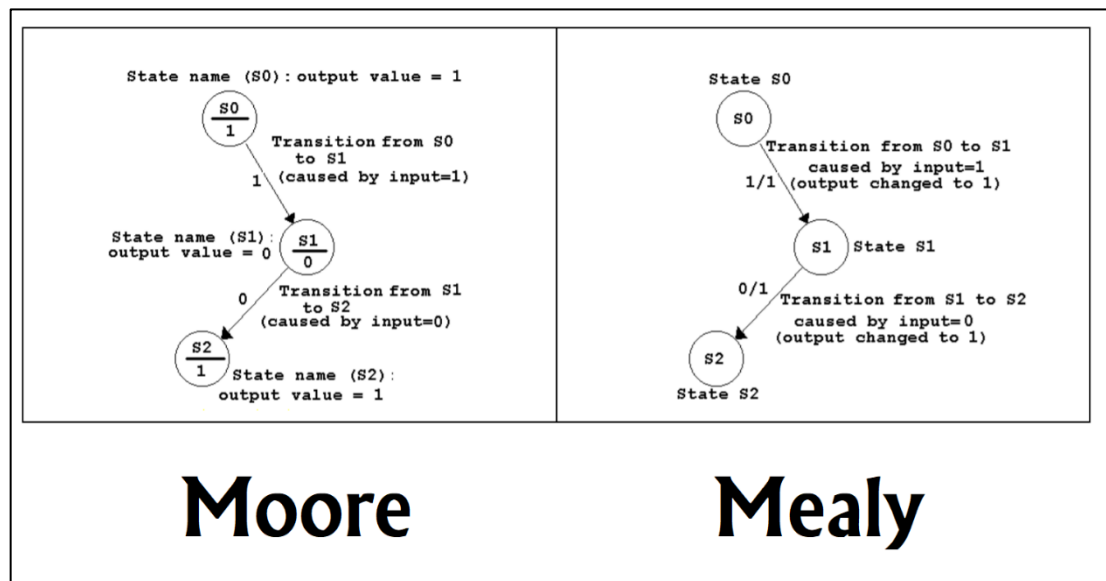
אם בונים את הדיאגרמה נכון, תוך התייחסות לכל האפשרויות, וכותבים את הקוד אחד לאחד לפי הדיאגרמה, בלי קיצורי דרך, יש לנו "תעודת ביטוח" שלא תהיינה טעויות. יש כלים שממירים בצורה אוטומטית דיאגרמות מצבים ל-VHDL, אבל בפועל המהנדסים בתעשייה כרגיל לא משתמשים בהם.

מכונות MOORE מול מכונות MEALY

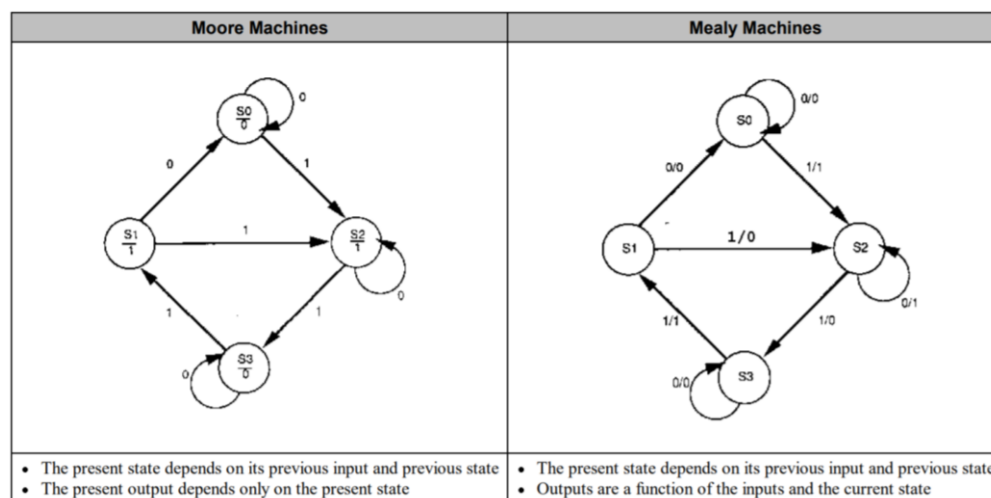
שני סוגי מכונות המצבים הנפוצים, נקראים על שם שני מדענים בולטים בתחום, שהמציאו אותם: Edward Forrest Moore ו-George H. Mealy. כדי להסביר את ההבדל ביניהן, נגדיר מושגים בסיסיים הנוגעים לשני סוגי מכונות המצבים:

INPUT	Input signal causing changes in the machine status
OUTPUT	Output signals coming out of the machine, based on its predefined behavior
STATE	One of the predefined steps in the operation of the machine
TRANSITION	A change from state to state
INITIAL STATE	The state that the machine wakes into, upon RESET

לפי מושגים אלה, יובן תרשים ההסבר הבא (שמתאר חלק מתוך תרשים, כל פעם באחת מהשיטות הללו), וההסבר שיבוא בעקבותיו:



ההבדל העקרוני הוא מה קובע את ה-OUTPUT. במכונות MOORE, מה שקובע את ה-OUTPUT הוא המצב (STATE), ולכן ה-OUTPUT נכתב בתחתיתו של כל מצב¹¹. על חיצוי המעבר (TRANSITION) מופיעים רק ה-INPUTS המתקבלים, וגורמים את המעברים השונים בין המצבים. במכונות MEALY, מה שקובע את ה-OUTPUT הוא המעבר (TRANSITION) הספציפי בין שני המצבים הספציפיים שלפני ואחרי המעבר. לכן ה-OUTPUT ייכתב על החץ שמסמל את המעבר. המספר השמאלי שבצמוד המספרים שצמוד לחיצים הוא ה-INPUT שגורם את המעבר, והימני הוא ה-OUTPUT שנגרם בעקבות המעבר. בתרשים הבא מופיעה דוגמה נוספת להשוואה בין סוגי המכונות. הפעם מתוארות המכונות בשלמותן, ולא רק קטעים מהתרשים:



שימו לב לדוגמה, שה-OUTPUT במעבר ל-S2 בתרשים הימני תלוי בשאלה מאיזה מצב הגענו אליו, בשונה ממה שמופיע בתרשים השמאלי. נראה עתה תיאור של מכונת מצבים ב-VHDL, גם היא מסוג MOORE, באופן שונה מסגנון התכנון הקודם שראינו לפני כן:

¹¹ יש כאלה שמציינים אותו סמוך לעיגול של המצב.


```

library IEEE;
use IEEE.std_logic_1164.all;

entity MOORE is
port
(
    CLK: in std_logic;
    RESET: in std_logic;
    X: in std_logic;
    Z: out std_logic
);
end MOORE;

architecture MOORE_ARC of MOORE is
    type StateType is (S0, S1, S2, S3);
    signal CURRENT_STATE: StateType;
    signal NEXT_STATE: StateType;
begin
    -- Synchronous process
    SYNCH: process
    begin
        wait until rising_edge(CLK);
        if (RESET = '1') then
            CURRENT_STATE <= S0;
        else
            CURRENT_STATE <= NEXT_STATE;
        end if;
    end process;

    --Combinational process
    COMBIN: process(X, CURRENT_STATE)
    begin
        case CURRENT_STATE is
            when S0 =>

```

```

            Z <= '0';
            if (X = '0') then
                NEXT_STATE <= S0;
            else
                NEXT_STATE <= S2;
            end if;
        when S1 =>
            Z <= '1';
            if (X = '0') then
                NEXT_STATE <= S0;
            else
                NEXT_STATE <= S2;
            end if;
        when S2 =>
            Z <= '1';
            if (X = '0') then
                NEXT_STATE <= S2;
            else
                NEXT_STATE <= S3;
            end if;
        when S3 =>
            Z <= '0';
            if (X = '0') then
                NEXT_STATE <= S3;
            else
                NEXT_STATE <= S1;
            end if;
        end case;
    end process;
end MOORE_ARC;

```

הסגנון הזה משתמש בשני משתנים שכוללים מצב – אחד את המצב הנוכחי ואחד את המצב הבא. היא כוללת שני processes – אחד דואג להגדיר את המצב הבא ואחד דואג שבכל עליית שעון המצב הנוכחי יהיה המצב שהוגדר במחזור שעבר כ'המצב הבא'.

יש לציין, שהחלוקה למצב נוכחי ומצב הבא נובעת משיקולי נוחות, ולא קשורה לשוני בין MEALY ל-MOORE. מצורפת כאן דוגמא של מכונה בשיטת Mealy, שמכילה שני משתנים כאלה:

```

library IEEE;
use IEEE.std_logic_1164.all;

entity MEALY is
port(
    CLK: in std_logic;
    RESET: in std_logic;
    X: in std_logic;
    Z: out std_logic
);
end MEALY;

architecture MEALY_ARC of MEALY is
    type StateType is (S0, S1, S2, S3);
    signal CURRENT_STATE: StateType;
    signal NEXT_STATE: StateType;
begin
    -- Synchronous process
    SYNCH: process
    begin
        wait until rising_edge(CLK);
        if (RESET = '1') then
            CURRENT_STATE <= S0;
        else
            CURRENT_STATE <= NEXT_STATE;
        end if;
    end process;

    --Combinational process
    COMBIN: process(X, CURRENT_STATE)
    begin
        case CURRENT_STATE is
            when S0 =>
                if (X = '0') then

```

```

                Z <= '0';
                NEXT_STATE <= S0;
            else
                Z <= '1';
                NEXT_STATE <= S2;
            end if;
        when S1 =>
            if (X = '0') then
                Z <= '0';
                NEXT_STATE <= S0;
            else
                Z <= '0';
                NEXT_STATE <= S2;
            end if;
        when S2 =>
            if (X = '0') then
                Z <= '1';
                NEXT_STATE <= S2;
            else
                Z <= '0';
                NEXT_STATE <= S3;
            end if;
        when S3 =>
            if (X = '0') then
                Z <= '0';
                NEXT_STATE <= S3;
            else
                Z <= '1';
                NEXT_STATE <= S1;
            end if;
        end case;
    end process;
end MEALY_ARC;

```

ב-case שבדוגמה זו אין את האופציה when others שעליה למדנו בסמסטר הקודם, כי כיסינו בצורה מפורשת את כל הערכים שהמשתנה CURRENT_STATE יכול להכיל.

כללים לכתיבה נכונה של מכונות מצבים ב-VHDL באמצעות enumerated type

הערה: אפשר להשתמש במשתנה זה לשימושים שונים שאינם בדיוקא מכונות מצבים.

- הגדרת סוג חדש של enumerated type נעשית ב-syntax הבא (מילים שמורות באדום):

```
type type_name is (type_element1, type_element2, ...);
```

דוגמאות:

```
type FSM_TYPE is (S0, S1, S2, S3, S4);
signal STATE: FSM_TYPE;
```

```
type StateType is (IDLE, COIN1, COIN2);
signal CURRENT_STATE, NEXT_STATE: StateType;
```

```
type Weekday is (SUNDAY, MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY);
signal DAY: Weekday;
```

```
type CONTROL_STATE is (IDLE, DO_READ1, DO_READ2, DO_READ3, DO_WRITE1, DO_WRITE3,
DO_WRITE3);
signal STATE: CONTROL_STATE;
```

- לכל משתנה מסוג enumerated type, יש להגדיר אלו ערכים הוא יכול לקבל, בלי לחזור יותר מפעם אחת על ערך מסוים.
דוגמה של טעות (חזרה פעמיים על S0):

```
--The following type is illegal:
type BAD_TYPE is (S0, S1, S2, S0);
```

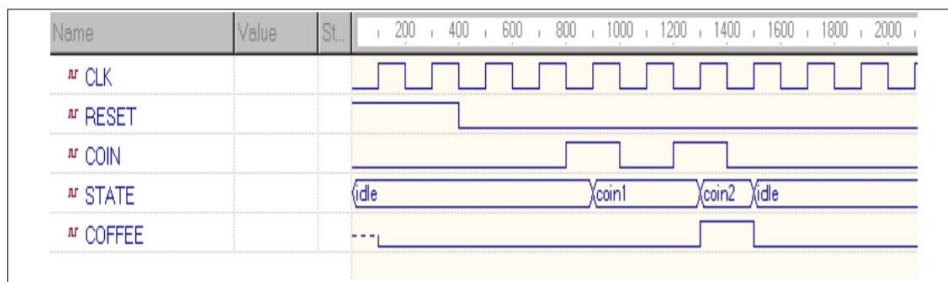
עם זאת, שני סוגים שונים של enumerated type יכולים לקבל ערך דומה.
דוגמה (S0 יכול להופיע ברשימות של שני סוגי המשתנים):

```
type THIS_IS_OK is (S0, S1, S2);
type THIS_IS_OK2 is (S0, S00, S000, S0000);
```

- עבור משתנה מהסוג הזה שהגדרנו בעצמנו, אין צורך בהתייחסות ל-when others, כל עוד התייחסנו בקוד לכל המצבים האפשריים שהגדרנו, בגלל שהוא לא יכול להכיל מצב אחר ממה שהגדרנו.

סימולציה של מכונת מצבים

כאן מוצגת סימולציה של מכונת המצבים שראינו, כמו סימולציה של כל תכנון שראינו עד כה:



שימו לב שב-STATE לא נראה מספרים או אותות לוגיים כלשהם, אלא את השמות המוכרים שהגדרנו.

השיטות השונות (קידודים שונים) לייצוג מצבים של מכונת מצבים באמצעות Flip-Flops

אחרי סינתזה, המצבים השונים ייוצגו בתור רצפים של 1 ו-0 (standard logic vector), שיישמרו בסדרת Flip-Flops. במילים אחרות, המצב של המערכת בכל רגע נתון, יהיה שמור בתוך סדרת Flip-Flops בתור מספר בינארי שמייצג את המצב הספציפי. ישנן כמה דרכים לבטא מספרי מצבים (ובכלל מספרים במערכת ספרתית). כאן מוצגות שיטות הקידוד הקלאסיות, בהדגמה של מכונה בעלת 8 מצבים (שני הראשונים הם היותר קלאסיים):

	Binary (Sequential)	One-hot	Gray	Johnson
	successive radix 2	One bit per state. Only 1 bit is set per a given state.	radix 2 encoding Only 1 bit changes in transition of consecutive states	consecutive states are based on shifted values, where the LSB is an invert of the MSB of the previous state
S0	"000"	"00000001"	"000"	"0000"
S1	"001"	"00000010"	"001"	"0001"
S2	"010"	"00000100"	"011"	"0011"
S3	"011"	"00001000"	"010"	"0111"
S4	"100"	"00010000"	"110"	"1111"
S5	"101"	"00100000"	"111"	"1110"
S6	"110"	"01000000"	"101"	"1100"
S7	"111"	"10000000"	"100"	"1000"
	3 bits	8 bits	3 bits	4 bits

הסבר מפורט יותר :

- **Binary (Sequential)**¹² – ייצוג בינארי רגיל, כמו שאנחנו רגילים להציג מספרים בבסיס 2. בשיטה זו, עבור ייצוג N מצבים, נצטרך $\log_2(N)$ ביטים¹³.
- **One-Hot** – ייצוג המספר, כך שיהיה 1 במיקום התואם למספר, ו-0 בכל שאר המיקומים. בשיטה זו, עבור ייצוג N מצבים, נצטרך N ביטים.
- **Gray** – ייצוג די דומה לייצוג בינארי רגיל, אך בסדר אחר, כך שבכל מעבר ממספר למספר שאחריו (וכן מהמספר הגבוה ביותר חזרה ל-0), תשתנה רק ספרה אחת. בשיטה זו, עבור ייצוג N מצבים, נצטרך $\log_2(N)$ ביטים¹⁴. בגלל שבשיטה זו משתנה כל פעם רק ספרה אחת, הדבר מונע בעיות glitch שעלולות להתרחש בשינוי של כמה ביטים באותו מחזור.
- **Johnson** – ייצוג באמצעות סדרת אפסים עבור 0, והכנסת סדרה של ספרות 1 צמודות, ש'מטיילות' לכיוון שמאל, כך שהסדרה 'תיעלם' אחרי המספר הגבוה ביותר. בשיטה זו, עבור ייצוג N מצבים, נצטרך N/2 ביטים¹⁵.

היתרונות והחסרונות של כל אחת משיטות הקידוד מוצגות כאן :

	Binary (Sequential)	One-hot	Gray	Johnson
Area	Minimal area based on minimal number of Flip-flops	Flip-flops number is as many the total number of states	Minimal area based on minimal number of Flip-flops	Not that many Flip-Flops as in "one-hot"
Delay	Might produce long timing delays in the next-state-logic	Since one bit is set for a state next-state-logic is minimized. (short timing delays)	Successive Gray codes differ in one bit, thus small amount of next state logic is needed	Successive Johnson codes differ in one bit. (small amount of logic)

לסיכום הטבלה : מבחינת מיקום (מספר הביטים עבור ייצוג מצב), החסכוני ביותר להשתמש ב-Binary או Gray, אחריהם Johnson והכי בזבזני הוא One-hot. מבחינת ההשעייה (הנגרמת מלוגיקה מסובכת עבור מעבר ממצב למצב, המסתבכת כאשר יש צורך בשינוי יותר ספרות), החסכוני ביותר הוא Gray (שינוי ביט בודד), אחרי One-hot ו-Johnson (שינוי שני ביטים) והכי מועד לפרענות הוא Binary (לפעמים הרבה ביטים ישתנו במעבר). יש לסייג את העניין הזה : כי

¹² תרגום מילולי : בינארי סדרתי/עוקב.

¹³ אם מתקבל מספר לא שלם ב-log, מעגלים כלפי מעלה.

¹⁴ אם מתקבל מספר לא שלם ב-log, מעגלים כלפי מעלה.

¹⁵ אם N אי-זוגי, נצטרך (N+1)/2 ביטים, כשאחת מהאפשרויות לייצוג לא תהיה בשימוש.

לפעמים יש מעברים בין מצבים שאינם עוקבים (לדוגמה: מ-S3 ל-S1 בדיאגרמת הבועות האחרונה שראינו), ובהם בכל מקרה יהיה כנראה שינוי של כמה ביטים.

יש לציין, שלשיטת One-Hot יש יתרון במניעת טעויות: אם בגלל רעש אחת הספרות תשתנה (תופעה בשם SEU=Single Event Upset), הטעות תתגלה, כי תהיינה 2 ספרות '1' או שלא תהיינה כאלה כלל, מה שלא הגיוני בקידוד כזה. בשביל לממש את היתרון הזה, צריך להוסיף מנגנון גילוי שגיאות, שיגלה את הבעיה ויטפל בה.

מצד שני, אם אין מנגנון תיקון שגיאות, במקרה של Single Event Upset, עלול להיווצר רצף חסר משמעות. בעקבות כך, הלוגיקה תהיה תקועה, כי היא לא בנויה לרצף כזה. לא יעזור when others בשביל למנוע את הבעיה הזו, כי אין כלל others שמוגדרים כאפשרויות חוקיות שהמערכת מכירה בהן.

שבוע 7

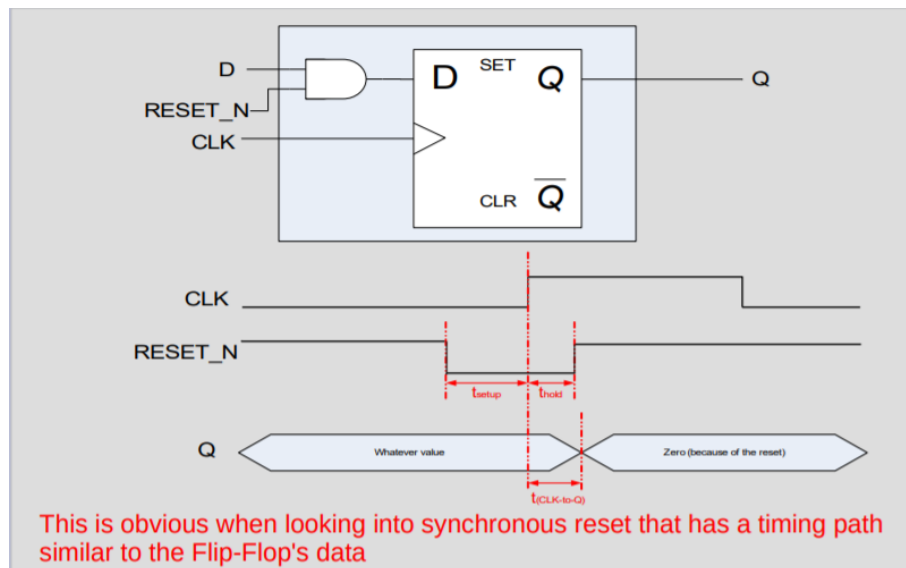
תכנון סינכרוני

התיכנון של רכיבים צריך להיות סינכרוני, כלומר: כל ה-FF יקבלו את אותו השעון. זה נצרך מאד כדי שנוכל להתייחס לנוסחאות הזמנים (setup ו-hold) שהכרנו בשבוע 4. אם המערכת תהיה לא סינכרונית, לא נוכל לדעת האם כל ה-FF "מתקשר" בצורה נכונה (מבחינת הזמנים) עם ה-FF האחרים, כי אין לנו נקודות ייחוס יציבות, שהזמנים בכל הרכיבים יכולים להימדד מולן.

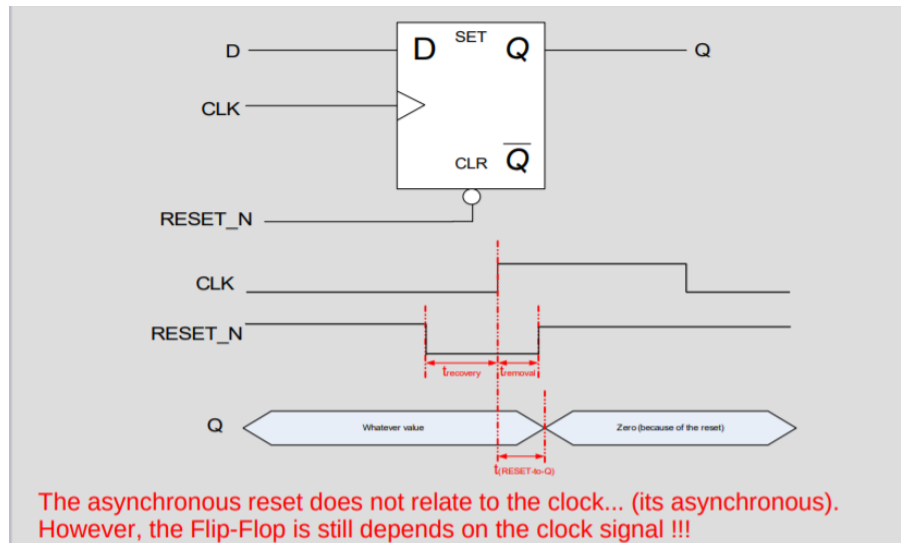
תכנון זמנים עבור אות RESET

גם אות RESET סינכרוני (שמתרחש בפועל רק בתחילת מחזור) צריך לעמוד בנוסחאות ה-setup וה-hold, כי גם הוא סוג של סיגנל.

בתרשים הבא מופיע החיבור בפועל של RESET סינכרוני. רואים שההכנסה שלו לתוך ה-FF היא באותו מסלול הכנסה של כל אות אחר, וממילא מובנת הדרישה שיעמוד בנוסחאות הזמנים.



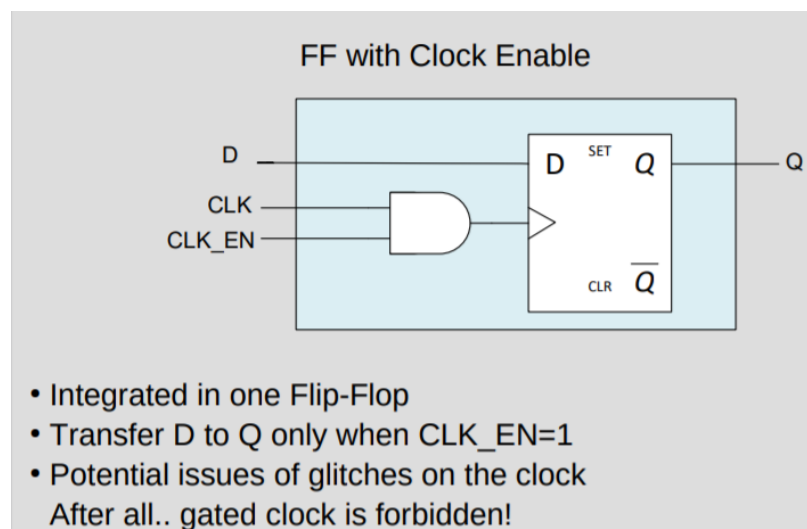
לעומת זאת, ב-RESET א-סינכרוני (שמתרחש בפועל מיד עם נתינת הפקודה) אי אפשר לדבר על עמידה בזמנים הנ"ל, כי הוא לא כפוף כלל לשעון. בתרשים הבא מתואר החיבור של RESET א-סינכרוני לתוך ה-FF. מתוך התרשים נראה שהאות הזה אינו כמו שאר אותות המידע הזורמים בין חלקי המערכת בכפוף לשעון.



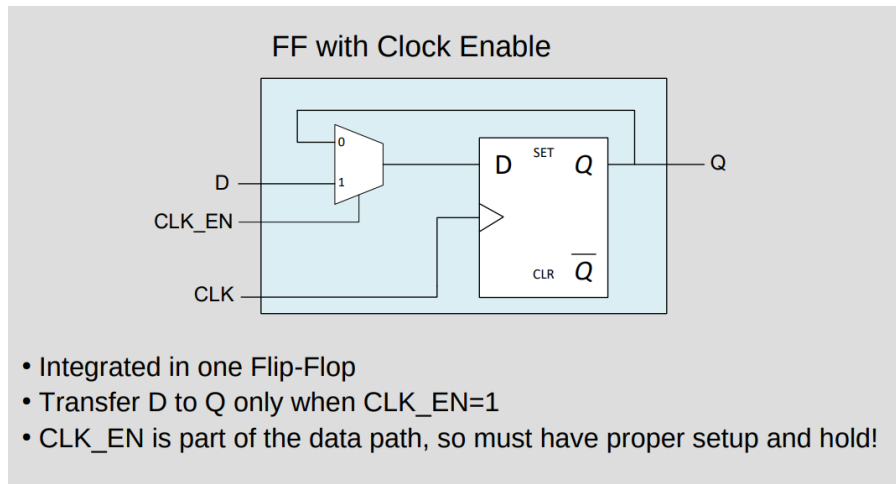
עם זאת, גם ל-RESET א-סנכרוני יש זמנים שהוא צריך לעמוד בהם, והם נקראים "recovery time" (מקביל ל-setup) ו-"removal time" (מקביל ל-hold). בנוסף, יש במקרה זה פרק זמן בשם "reset-to-Q", שמקביל לזמן "clock-to-Q" שראינו לפני כן. זהו הזמן שעובר מכניסת אות ה-RESET ל-FF ועד שהערך של Q הוא הערך המתאים למצב RESET.

מימוש clock enable ללא בעיות זמנים

לפעמים רוצים להגביל את ההשפעה של השעון על FF מסוים, ולתת לו את ההשפעה רק כאשר כניסה בשם "clock enable" נמצאת ב-1 לוגי. אפשר לממש את הרעיון הזה בצורה המופיעה בתרשים הבא:



אבל התכנון הזה – שבו השעון עובר דרך לוגיקה בדרך אל ה-FF – מאד לא מומלץ, כי בלוגיקה קומבינטורית נוצר פעמים רבות glitch, שמשבש את חישובי נוסחאות הזמנים, וכן ליצור כאילו 'עליות שעון' נוספות שמקורן ב-glitch ולא בשעון המערכת. הדרך הנכונה ליצירת clock enable היא בדרך הבאה:



ה-*enable* יושב בדרך של המידע המגיע ל-FF או יוצא ממנו, ולא בחיווט של השעון.

דגשים נוספים לתכנון סינכרוני מיטבי

בשקף הבא (האחרון במצגת "מסלולי זמנים בתכנון דיגיטלי" שבמודל באזור של שבוע 3), מנוסחים כמה כללים שיש להקפיד עליהם בתכנון:

Design Rules in Flip-Flops

It is so important to avoid Setup and Hold problems so...

- Make sure that the clock skew is minimal
- Make your design synchronous (don't use more than one clock domain)
- Don't use "inverted clock"
- Don't use "gated clock"
- Don't use "derived clock"
- Avoid using latches...
- And (not related to setup/hold timing).. Every Flip-Flop must have a reset signal that will bring him to a known state when the reset is asserted.

נסביר את הכללים בקצרה:

- יש להקפיד על *skew* מינימלי לשעון. כלומר, למנוע מצב שבו ה-FF השונים יקבלו את אות השעון בזמנים שונים בעקבות אורכי זמן *wire delay* שונים, שנגרמים מהפרש גדול באורך החוטים מהשעון המרכזי אל ה-FF השונים. כנראה נלמד בהמשך על מושג זה בפירוט.
- יש להשתמש בשעון אחד עבור כל המערכת.
- יש להימנע משימוש ב"שעון מהופך", כלומר אות שעון שמגיע ל-FF דרך שער NOT. יש בזה שתי בעיות: ראשית, יש להימנע מהעברת אות השעון דרך שערים לוגיים, כפי שכבר ראינו. אך חמור מכך: בגלל ההיפוך – ה-FF שיקבל את אות השעון שעובר דרך המהפך, יתחיל ויסיים את המחזור הלוגי שלו בזמן שעבור שאר ה-FF ייחשב כאמצע המחזור!
- אין להעביר אות שעון ל-FF דרך שער לוגי, כפי שראינו לעיל.
- יש להימנע מ-*derived clock*. כלומר, אין להעביר אות שעון ל-FF דרך יציאת Q של FF אחר. זה מאריך את ה-*delay* בהגעת האות אל ה-FF הספציפי, מה שעלול לגרום לבעיית *skew* חריפה.

g ← A

j' ← j

צריך שאות השעון יגיע לכל ה-FF מרשת חוטים ייעודית שיוצאת מהשעון המרכזי ומגיעה באותו תזמון לכל ה-FF.

- יש להימנע משימוש ב-latch.
- הסבר : FF מעביר מ-D ל-Q את הערך שהיה ב-D ברגע עליית השעון, ומכאן והלאה אין מעבר של מידע מ-D ל-Q כל עוד לא התרחשה עליית שעון נוספת. לעומת זאת, latch ממשיך להעביר מידע מ-D ל-Q כל עוד השעון נמצא ב-1 לוגי. המידע יפסיק לעבור רק עם ירידת השעון.
- השימוש ב-latch מזמן בעיות, כי לא תמיד אפשר לדעת מה יהיה בתוך D לכל אורך חצי המחזור שבו השעון נמצא ב-1 לוגי, ואין שליטה על זמני ה-hold וה-setup של המידע הזה.
- לכל FF צריכה להיות כניסת RESET, שמביאה את ה-FF למצב ידוע בהינתן פקודת RESET. יש להקפיד על פולאריות אחידה (כולם פעילים ב-1 או כולם ב-0). כמו כן, צריך להקפיד שכל ה-FF יהיו או כולם בעלי RESET סינכרוני או כולם בעלי RESET א-סינכרוני.

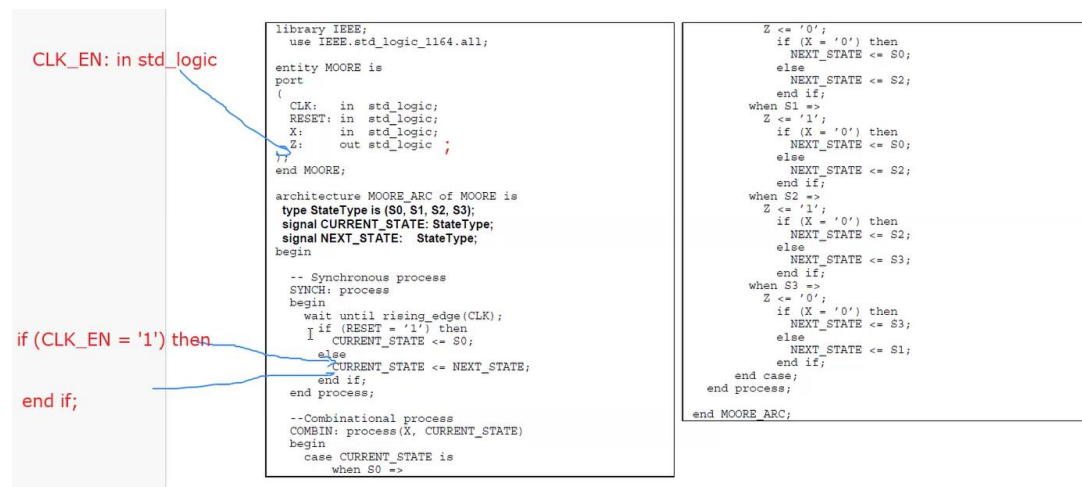
שימוש ב-clock enable במכונת מצבים

לפעמים נהיה מעוניינים שמכונת המצבים תוכל לעבור ממצב למצב רק בחלק ממחזורי השעון. השורות שיש לחוסיף בשביל השילוב של clock enable בקוד ה-VHDL של מכונת המצבים, תלויות בסוג התכנון. *כאן מתוארת תוספת השורות לקוד מכונת מצבים המבוסס על process אחד :

<pre>library IEEE; use IEEE.std_logic_1164.all; entity COFFEE_MACHINE is port (CLK: in std_logic; RESET: in std_logic; COIN: in std_logic; COFFEE: out std_logic; CLK_EN: in std_logic;); end COFFEE_MACHINE; architecture MOORE of COFFEE_MACHINE is type StateType is (IDLE, COIN1, COIN2); signal STATE: StateType; begin P0: process begin wait until rising_edge(CLK); if (RESET = '1') then STATE <= IDLE; COFFEE <= '0'; else if (CLK_EN = '1') then case STATE is when IDLE => if (COIN = '1') then</pre>	<pre>STATE <= COIN1; COFFEE <= '0'; end if; else STATE <= IDLE; COFFEE <= '0'; end if; when COIN1 => if (COIN = '1') then STATE <= COIN2; COFFEE <= '1'; else STATE <= COIN1; COFFEE <= '0'; end if; when COIN2 => if (COIN = '1') then STATE <= COIN1; COFFEE <= '0'; else STATE <= IDLE; COFFEE <= '0'; end if; end case; end if; end process; end MOORE;</pre>
--	---

ההוספה כוללת :

- כניסה בשם EN_CLK שצריכה להיות מוצהרת ב-entity.
 - התניה של השורות שמבצעות מעבר ממצב למצב באמצעות השורה if (CLK_EN = '1') then.
 - שורת end if לאחר השורות שמבצעות מעבר ממצב למצב.
- *כאן מתוארת תוספת השורות לקוד מכונת מצבים המבוסס על שני processes :

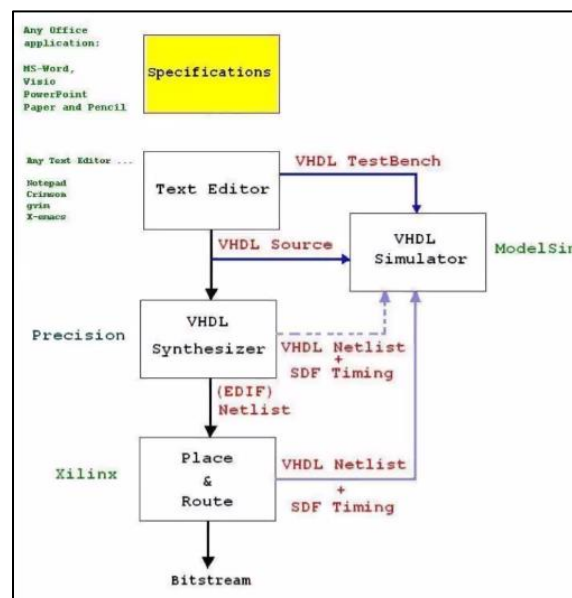


ההוספות במקרה הזה די דומות למקרה הקודם, אך התנאי ושורת התנאי צריכות לבוא בתוך ה-process הסינכרוני (הקצר יותר) שהוא זה שאחראי 'הלכה למעשה' למעבר בין המצבים.

שבוע 8

הצורך בסימולציה Gate level simulation

ניזכר בתרשים של תכנון HDL שראינו בתחילת הקורס (ובקורס הקודם):



עד עכשיו למדנו על סימולציה שנעשית על קוד VHDL (מיוצג בתרשים על ידי החץ שנכנס ל- VHDL simulation מכיוון שמאל). אך אחרי שלבי הסינתזה ו-place & route התכנון עובר המרה לצורה חדשה, ועליה לא עשינו עד היום סימולציה (הסימולציה הזו מיוצגת בתרשים על ידי 2 החיצים התחתונים הנכנסים ל-VHDL simulation).

יש צורך לעשות סימולציה לתכנון בשלבים האלו, בנוסף לזו שעשינו לקוד ה-VHDL, כי מה שנכנס לסיליקון הוא העיבוד אחרי סינתזה ו-P&R, וצריך לוודא ששלבי העיבוד לא פגעו בתכנון. לפעמים תכנון גבולי ב-VHDL יתפרש לא נכון בכלי הסינתזה. כמו כן, עלולות להיות בעיות בכלי הסינתזה. טעויות כאלה עלולות להיווצר, וחבל מאד שיתגלו רק בשלב ההטענה לסיליקון.

הסימולציה הזו נקראת "Gate level simulation". הסימולציה אחרי סינתזה נקראת "Post Synthesis", וזו שנעשית אחרי Place & Route נקראת "Post P&R" או "Post Implementation". עבור FPGA מספיק לעשות סימולציה אחרי ה-P&R, כי המעבר מסינתזה ל-P&R ומשם לרכיב הוא מהיר ("מרחק לחיצת כפתור"), ואם תהיינה בעיות נוכל לחזור בקלות לסינתזה ולבדוק אותה. אבל ב-ASIC ייקח הרבה זמן עד שנקבל את ה-P&R, ולכן אי אפשר לוותר על סימולציה אחרי הסינתזה. יש לציין שלפעמים ה-testbench יצטרך לעבור שינוי כדי להתאים לסימולציה הזו.

הקבצים הנדרשים עבור סימולציה ברמת gate level (כולל הרחבה על SDF)

כדי לבצע סימולציה כזו יש צורך בשני קבצים (המופיעים בתרשים לעיל):

- VHDL Netlist – אחראי לתיאור הפונקציה של התכנון, כלומר המבנה של המערכת, ותיאור יחסי הקלט והפלט. הוא כתוב בשפת VHDL¹⁶.

- SDF Timing – מתעסק עם נושא הזמנים (SDF = Standard Delay Format). הוא כתוב בשפה ייעודית, שגם היא נקראת SDF. יש אפשרות שלא להשתמש בו, אם מעוניינים שלא לבדוק את נושא הזמנים (סימולציה מסוג "without timing").

כלי הסינתזה וגם ה-VIVADO יודעים לתת לנו את שני הקבצים האלה. יש לציין שקובץ ה-SDF שנותן כלי הסינתזה אינו מדויק לחלוטין, כי כל עוד לא נעשה P&R אי אפשר לדעת באמת את ההשהיות. נציג כאן דוגמה לקובץ SDF (הוא נמצא ב-moodle):

```
Tips-SDF-format.txt
=====
Tip for understanding some of the SDF (Standard
Delay Format) syntax:
=====
Here is an example:

A Flip-Flop cell has different delays of clk-to-q,
as specified by the Silicon
vendor. It is not always the same delay:

when Q changes from 0 to 1:
22 ps if the Silicon is "Best Case" ("min")
28 ps if the Silicon is "Typical" ("typ")
33 ps if the Silicon is "worst Case" ("max")

when Q changes from 1 to 0:
25 ps if the Silicon is "Best Case" ("min")
30 ps if the Silicon is "Typical" ("typ")
37 ps if the Silicon is "worst Case" ("max")

In order to describe these 6 values (two
triplets... which are altogether
6 values!!!) the syntax is...

(CELL (CELLTYPE "DFF")
(INSTANCE dut)
(DELAY
(ABSOLUTE
(IOPATH (posedge clk) q (22:28:33) (25:30:37))
)
)
)
```

הסבר הקובץ "מלמעלה"¹⁷:

רוב השורות הן הערות הסבר באנגלית, ורק 8 השורות התחתונות הן ה-syntax עצמו. מדובר על FF מסוג D, שלציאה שלו (Q) יש delay במעבר מ-0 ל-1, וגם במעבר מ-1 ל-0. במעבר מ-0 ל-1 ערכי ה-delay הם: מינימלי – 22 פיקו-שנייה, טיפוסי – 28 פיקו-שנייה, מקסימלי – 33 פיקו-שנייה. במעבר מ-1 ל-0 ערכי ה-delay הם: מינימלי – 25 פיקו-שנייה, טיפוסי – 30 פיקו-שנייה, מקסימלי – 37 פיקו-שנייה.

Global Reset/Power On Reset/Reset On Configuration (ROC)

ישנם תכנונים שמבצעים איפוס של כל ה-Flip-Flops עם טעינת התכנון לציפ, ובהתאם מתבצעת כך גם הסימולציה.

¹⁶ בהמשך נדבר על כך שהרבה פעמים נשתמש בקבצים בשפת Verilog.
¹⁷ לא נכנסנו ללימוד הפרטים של השפה במסגרת ההרצאה.

היתרון של זה הוא שלא צריך לבצע RESET באופן יזום, וכן יודעים מלכתחילה מה הערכים בכל ה-FF בתחילת ההפעלה. אך יש לזה חסרון במקרים שונים. לדוגמה: אם עושים סימולציה על תכנון שמשמש בקידוד One-Hot¹⁸ כדי לייצג מספרים, זה ייצור בעיה, משום ששורת אפסים (שנוצרת ב-RESET האוטומטי) אינה ייצוג חוקי של אף מספר בקידוד הזה. יש דרך לגרום לכך שביטים מסוימים לא יתאפסו בהדלקה, אבל צריך לזכור את הנקודה הזו, ולטפל בה.

בעקבות תכונה זו, יש לזכור שלא לנסות לבצע פעולות ב-testbench בפרק הזמן שבתחילת סימולציית ה-gate level, כי מבחינת הסימולטור אין אפשרות לעשות כלום כל עוד לא הושלמה הקונפיגורציה.

סימולציה מעורבת VHDL ו-Verilog

יצרני הסיליקון עובדים כיום עם קובצי netlist ב-Verilog ולא ב-VHDL. זה אומר שגם VIVADO יוציא קובץ netlist ב-Verilog. ולכן צריך להתרגל לכך שסימולציה ברמת gate-level של תכנון בשפת VHDL, צריך לעשות בשילוב של שפת VHDL ושפת Verilog.

הדגמת סימולציית Gate level simulation

ראינו בשיעור הדגמה של Gate level simulation, ב-VHDL וכן סימולציה של שילוב בין VHDL לבין Verilog. ב-moodle (ביחידה של מטלה 6) נמצאים קבצי ההסבר והקבצים לשימוש, שבהם השתמשנו בשיעור.

יש לציין שסימולציית Gate level simulation לא קיימת בגירסה החינמית של Modelsim.

תופעת ה"אקורדיון"

בסימולציה ראינו דבר חשוב שרואים רק בסימולציה ברמת gate level: סימלצנו מונה יורד שאמור להוציא ב-TC אות של 1 לוגי כאשר הוא מתאפס, וראינו שנוצר glitch (עלייה קטנה שקרתה שלא בהתאם לתכנון המקורי של המערכת), בגלל שכל ביט משתנה בזמן אחר (כל FF עם ההשהיה והמיקום שלו), מה שיוצר ערך 0 בתוך המונה למשך זמן קצר במעבר בין מספרים. הדוגמה שראינו הייתה במעבר בין 2 ל-1, וכן בין 4 ל-3. נתמקד בדוגמה השנייה (מעבר בין 4 ל-3):

	הערך המיוצג	הביטים שמייצגים את המספר					
		bit 4	bit 3	bit 2	bit 1	bit 0	
	4	0	0	1	0	0	מספר מקורי
Glitch!	0	0	0	0	0	0	החלפת ביט 2
	2	0	0	0	1	0	החלפת ביט 1
	3	0	0	0	1	1	החלפת ביט 0

ההחלפה היא מאד מהירה ביחס לקצב השעון, אך עדיין מספיק להיווצר glitch, כי הפיזיקה של הלוגיקה הקומבינטורית מהירה יותר מקצב השעון.

תופעה זו מכונה "אקורדיון", כי המספר גדל וקטן בערך מהיר, עד ההתייצבות (כמו שרואים בטבלה בטור "הערך המיוצג"), בדומה לאקורדיון שנמתח ומתכווץ בזמן הנגינה בו.

אפשר למנוע את התקלות שעלולות לבוא בעקבות תופעה זו, אם נדאג שיהיה Flip-Flop ביציאת ה-TC מהמונה. כך, יינתן בכל מחזור ביציאת ה-FF רק את הערך הסופי שהתקבל אחרי כל ה"אקורדיון". כמובן שיהיה צריך לדאוג לקיום משוואות הזמנים, כדי שהערך שייכנס ל-FF יהיה יציב, וכך גם היציאה שלו תהיה אמינה.

התופעה הזו מלמדת אותנו את אחת הסיבות לחשיבות של סימולציית gate level.

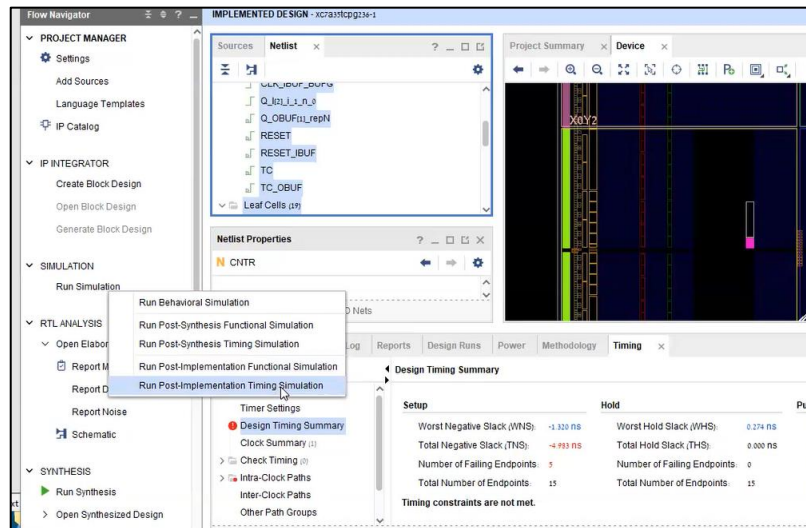
הוצאת קבצי SDF ו-nelist מ-VIVADO

כדי להוציא את הקבצים הללו מ-VIVADO יש:

- לבצע P&R ב-VIVADO.
- ללחוץ בתפריט הימני על SIMULATION.
- ללחוץ על Run Simulation.

¹⁸ שלמדנו מוקדם יותר, בהרצאה על מכונות מצבים.

- לבחור בחלון שנפתח את האופציה האחרונה, כמו בצילום כאן :



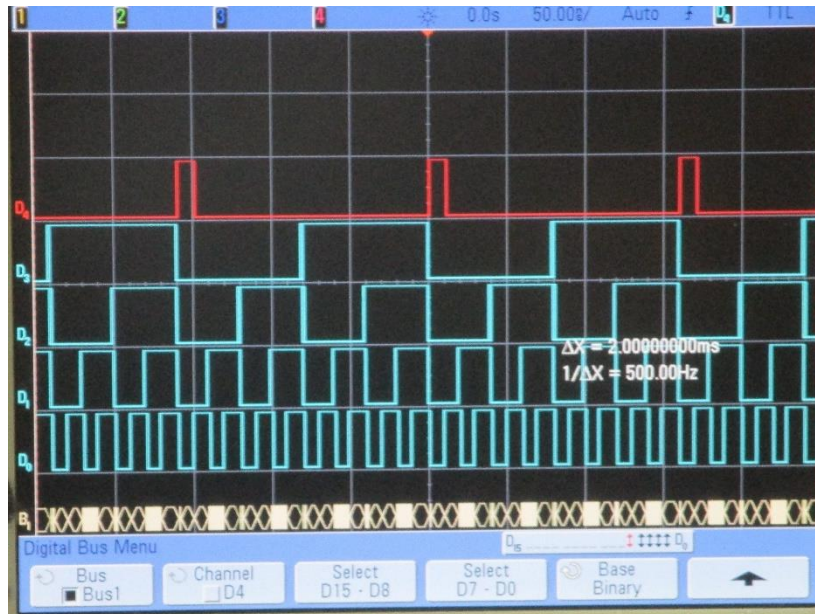
- בתיקייה של הפרויקט, תהיה תיקיה עם סיומת sim. שבתוכה יש להיכנס לעוד תיקיה בתוך תיקיה עד שנפתחת רשימת קבצים. בין הקבצים יהיו שני קבצים עם הסיומות : time_impl.sdf ו-time_impl.v הראשון הוא ה-SDF והשני הוא ה-netlist.
- צריך לשים את הקבצים הללו בתיקייה אחת יחד עם ה-testbench כדי לעשות סימולציה gate level ב-modelsim.

הצגת האותות הפנימיים של ה-FPGA באמצעות Logic Analyzer (מר ערן שלום)

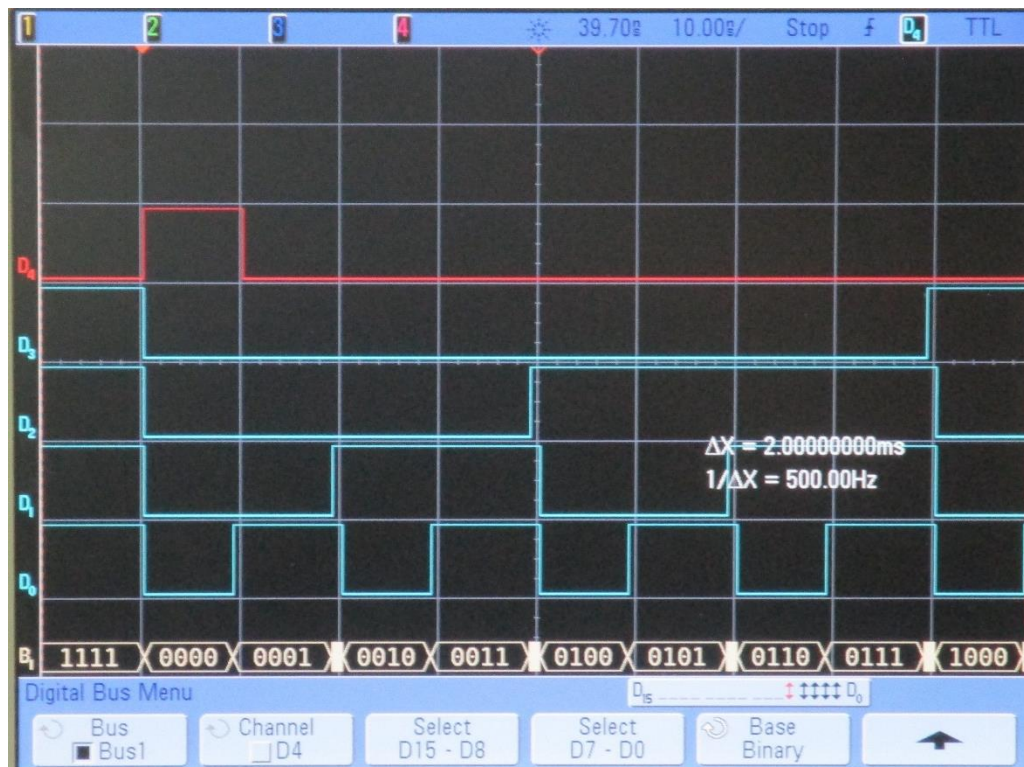
ראינו בשיעור הצגה של אותות ה-OUTPUT של רכיב ה-FPGA. הוצאנו את האותות לפינים של הרכיב באמצעות הנחיות מתאימות בקובץ ה-XDC, וחיברנו פרובים של ה-Logic Analyzer ליציאות ה-PMOD שעל הלוח, שמחוברות לפינים של ה-FPGA שברנו. האותות הם אות TC (בצבע אדום) ורביעיית הרגיסטרים של מונה בעל 4 בבחירת ביטים (הביטים מיוצגים כל אחד בפני עצמו בצבע ירוק), שעולה בספירה כל עליית TC.

ה-BUS (בצבע לבן בתחתית מסך ה-Logic Analyzer) הינו אפשרות לאיחוד 4 ביטים של המונה לתצוגה בינארית או הקסא-דצימלית.

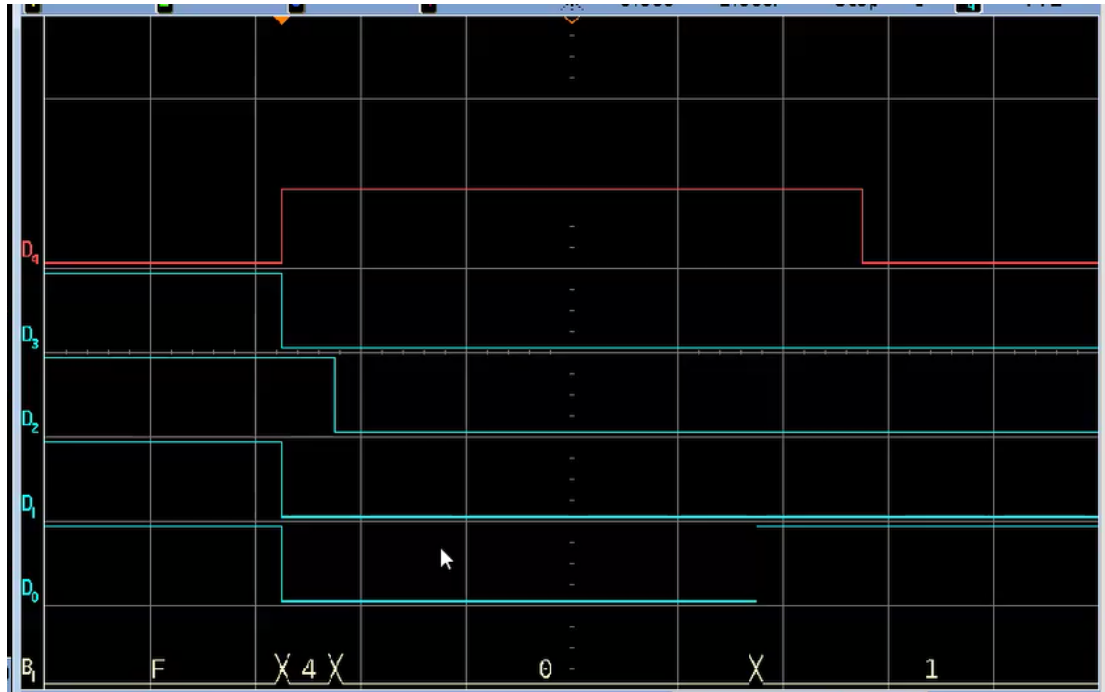
בתחילה לא חיברנו את ה-GROUND של הלוח ל-GROUND של ה-Logic Analyzer וראינו שייצוג האותות לא היה יציב. הדגשנו שצריך תמיד לחבר את ה-grounds של כל הכלים והרכיבים המשתתפים במערכת, כדי שתהיה למערכת הנמדדת ולמערכת המודדת נקודת ייחוס משותפת יציבה. לאחר חיבור ה-grounds קיבלנו אותות יציבים על המסך. האותות מוצגים כאן :



וברזולוציה גבוהה יותר בצילום הבא :



שימו לב שהסיפרה הנמוכה ביותר (LSB) משתנה בקצב מהר יותר, אחריה הספרה הבאה וכן הלאה עד הספרה הגבוהה (MSB) המשתנה בקצב האיטי ביותר. זה גורם לאות ה-BUS להראות כמה ערכים משתנים בפרק זמן קצר (בפועל זה נראה כמרובעים לבנים, כי אין מקום לייצג הרבה ערכים במיקום כל כך צר). זו תופעת ה"אקורדיון" שדיברנו עליה לפני כמה עמודים. כשמגדילים את הרזולוציה, אפשר לראות את ערכי ה"אקורדיון" שבמעבר בין מספרים :



שבוע 9

טיפול בבעיית זמנים (setup/hold) נתונה לאחר שהתגלתה

לאחר שכלי הסינתזה או כלי ה-Place & Route מגלים בעיית זמנים, מומלץ לפעול לפי סדר הפעולות הבא:

1. הבנה וזיהוי של בעיית הזמנים. כלומר, זיהוי הסיגנלים והמסלול (בקוד ה-VHDL), המעורבים בבעיה והבנת חומרתה ואופייה של הבעיה (VIVADO מפרט את מסלולי הבעיות כשלוחצים על האזהרות שהוא כותב).
2. לפעמים אפשר לתת לבעיה פטור (to give a waiver). למשל, אם הבעיה משפיעה על רכיב שאינו כפוף לשעון של המערכת שאנחנו מתכננים, אלא רכיב חיצוני, אפשר להתעלם מהבעיה, וכך לגמור את הטיפול. במקרה זה, הבעיה אינה באמת בעיה, כי רכיב שאינו כפוף לשעון המערכת אינו צריך לקיים את נוסחאות ה-setup וה-hold.
3. אם הבעיה משפיעה על רכיב שכפוף לשעון, ולכן אי אפשר לתת לה פטור, נבדוק האם אפשר לשנות את משהו מחוץ לרכיב ה-FPGA (למשל, לשנות משהו ב-Board או את תדר השעון).
4. אם אי אפשר לשנות מחוץ לרכיב ה-FPGA, נבדוק האם אפשר לשנות את ה-RTL¹⁹ (שינוי בקוד ה-VHDL) כך שמסלול הזמנים בו קיימת הבעיה ישתנה, כך שהבעיה לא תתקיים. לדוגמה: אפשר לפעמים להכניס FF בתוך המסלול, כך שהתהליך הלוגי של המסלול ייפרס על פני שני מחזורי שעון, ויעמוד בנוסחאות הזמנים.
5. אם אין אפשרות לשנות את ה-RTL, ננסה לעשות אופטימיזציה חזקה יותר בכלי הסינתזה או בכלי ה-Place & Route. ב-Vivado קובעים את רמת המאמץ והאופטימיזציה של הכלי באמצעות שינוי הגדרות בחלון ה-settings.
6. אם כל זה לא עוזר, אפשר לנסות למצוא גרסה יותר מתקדמת של כלי הסינתזה, או כלי סינתזה אחר טוב יותר.

¹⁹ראשי תיבות של: Register Transfer Level

7. לעיתים, ניתן להשפיע על מאמץ האופטימיזציה של כלי הסינתזה באמצעות "אילוץ יתר" (Over Constrain). הרעיון הוא לאלץ אילוץ זמנים שיגרום לבעיה להחריף (לדוגמה: להגדיל את תדר השעון בקובץ ה-XDC), וכך כלי הסינתזה או כלי ה-Place & Route יתאמצו יותר על מנת לפתור את הבעיה.

בדיקה והערכה (T&E = Test and Evaluation)

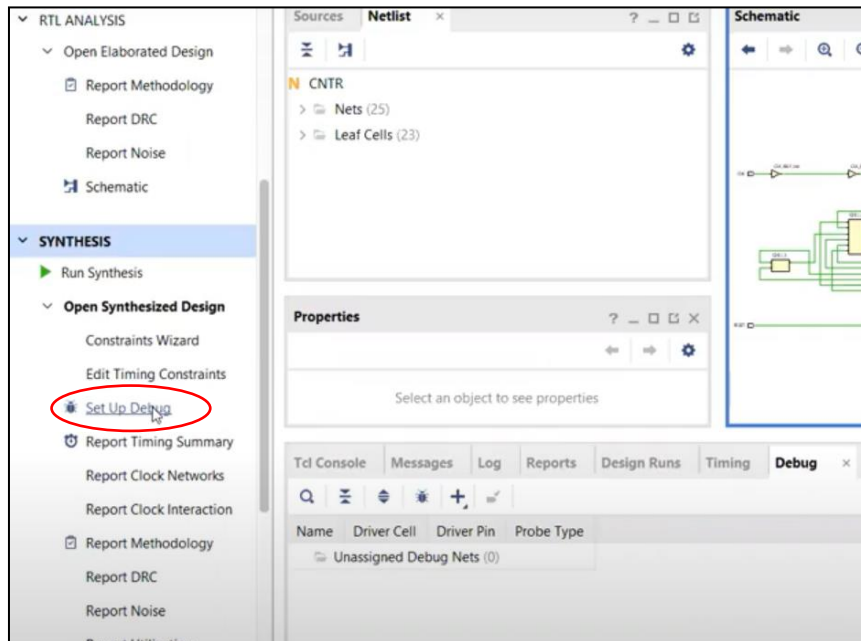
נושא זה נכון לכל תחום באלקטרוניקה, ובכלל בכל ייצור או פיתוח בעולם. חשוב לבדוק את המוצר בכל שלב בפיתוח/תכנון, לפני שמתגלות טעויות קריטיות כשכבר קשה לתקן, או כשהן כבר גרמו לנזק. ב-moodle (ביחידה של מטלה 6, תחת הכותרת "חומר רקע והסבר: מושגי יסוד בנושאי ניסויים ובדיקות") קיימים מצגות מפורטות שמסכמות את הנושא. כמה נקודות שלמדנו בשיעור:

- בדיקת "קופסה שחורה" היא בדיקה מבחוץ, האם הכלי פועל (ביציאותיו) בהתאם לכניסותיו, על פי הדרך שבה הוא אמור לעבוד. בדיקת "קופסה לבנה" היא בדיקה שבה רואים (ובחלק מהמקרים אף שולטים על) מה שקורה בתוך הכלי המתוכנן בזמן פעולתו.
- בתעשייה יש כוח אדם גדול מאד שמושקע על בדיקות וסימולציות.
- צריך לדאוג שהפיתוח יהיה בר-בדיקה. הנושא הזה נקרא "testability" או "בדיקותיות". לדוגמה: יצירת test mode, או הוספת נורת לד על הלוח שתשמש כסיוע לבדיקת פעולתו.
- בדיקת רגרסיה (נסיגה) היא בדיקה שנעשית במעבר מתכנון אחד לתכנון משוכלל יותר. צריך לבדוק גם שלא קילקלנו את מה שהיה בתכנון הקודם, וגם שמה שרצינו להשיג בשדרוג אכן הושג.

Chip-Scope / ILA

בסימולציות השונות, אפשר לראות את מצב האותות הפנימיים של התכנון בזמן הרצה. אך כאשר התכנון נמצא בתוך רכיב סיליקון, אין לנו גישה לאותות הפנימיים שבתוכו, אלא אם כן נקנה ציוד יקר מאוד שמאפשר גישה פיזית של מיקרו-פרובים לאותות הפנימיים. ל-Vivado יש כלי בשם Chip-scope²⁰, שמאפשר לנו לדגום אותות פנימיים בסיליקון לאחר טעינת התכנון לתוכו. ראינו בכיתה הדגמה לשימוש בכלי הזה, ונחזור כאן על עיקרי הדברים: כדי להשתמש בכלי הזה, יש לבצע סינתזה של התכנון ב-Vivado, וללחוץ אחר כך בסרגל הכלים השמאלי על "setup debug", שנמצא תחת הסינתזה.

²⁰ לחברות המתחרות יש גם כלים כאלה, בשמות אחרים.

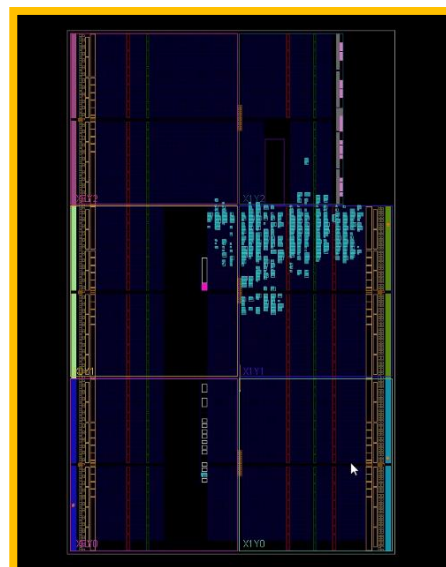


יפתח חלון שייתן לנו אפשרות לבחור אלו nets אנו רוצים לדגום. לאחר בחירת ה-nets, יהיה אפשר לבחור פרמטרים שונים כמו למשל גודל ה-buffer שבו יאוחסנו הדגימות (כלומר נקצה מקום לכמות הדגימות שהוא יעשה). כשה-buffer יתמלא, תפסיק הדגימה.

Vivado יוסיף לתכנון שני חלקים, ששם debug hub ו-Integrated Logic Analyzer (ILA). באמצעותם, האותות הפנימיים יועברו לרגליים חיצוניות של הרכיב, ויתאפשר לנו בהמשך לראות את האותות הפנימיים דרך הכבל המחובר בין הלוח למחשב.

אחרי הוספת החלקים האלה יהיה צריך לבצע סינתזה וגם P&R מחדש, למרות שהתכנון המקורי כבר עבר P&R, כי נוספו חלקים חדשים שעדיין לא עברו P&R. כמובן שבדרך ל-P&R יהיה צורך לשנות גם את קובץ ה-xdc, כי יש יציאות פעילות חדשות לרכיב. Vivado יפנה בעצמו לשינוי ה-xdc אחרי שנבקש ממנו לבצע P&R, וגם יעשה את השינוי בעצמו אם נאשר לו.

החלקים האלה, שנוספים לתכנון הקיים, תופסים לא מעט מקום ב-FPGA. בשתי התמונות המצורפות, יש השוואה של הדוגמה שראינו בכיתה, לפני תוספת החלקים האלה (תמונה שמאלית) ואחרי ההוספה (תמונה ימנית):



[אם התכנון המקורי תופס מקום רב, עלול להיווצר מצב שבו לא יישאר לנו מקום להוסיף את הכלי הזה, בפרט אם נרצה לדגום הרבה אותות פנימיים. כמו כן, מסלולי הזמנים עלולים להפוך ליותר בעייתיים]. אחרי טעינת התכנון החדש – שכולל את ה-Chip-Scope – לתוך ה-FPGA, ייפתח חלון שבו יהיה אפשר לראות את האותות. תהיינה שם מספר דגימות לפי גודל ה-buffer שבחרנו בשלבי ההכנה של ה-Chip-Scope. הדגימה מתבצעת ב"שידור חי", כשהלוח הפיזי עובד. אם נבצע RESET בצורה ידנית על הלוח, גם תהליך הדגימה יתחיל מחדש, יחד עם הפעלת הרכיב כולו מחדש. בהדגמה ראיו, שאפשר לבקש מהכלי לדגום את האותות משינוי מסוים (trigger) והלאה, באמצעות חלון ה-Trigger Setup.

Translator

ראינו בשיעור הדגמה של יצירת translator שמתרגם אות של שני ביטים פנימיים בתכנון, למילים שמביעות את הערך של המספר הבינארי שמופיע בשני הביטים. לדוגמה: המספר הבינארי 10 יתורגם ל-TWO. הפונקציה נכתבת בתוך ה-testbench. כל הקבצים שראינו בהדגמה מופיעים ב-moodle, ביחידה של מטלה 6.

הפונקציה מופיעה כאן:

```

18
19 function TRANSLATOR (X: std_logic_vector) return string is
20     variable X_V: std_logic_vector(1 downto 0);
21     variable S_V: string(1 to 5);
22 begin
23     X_V := X;
24     case X_V is
25         when "00" => S_V := " zero";
26         when "01" => S_V := " one";
27         when "10" => S_V := " two";
28         when "11" => S_V := "three";
29         when OTHERS => S_V := "all x";
30     end case;
31     return(S_V);
32 end TRANSLATOR;
33

```

הסבר קצר:

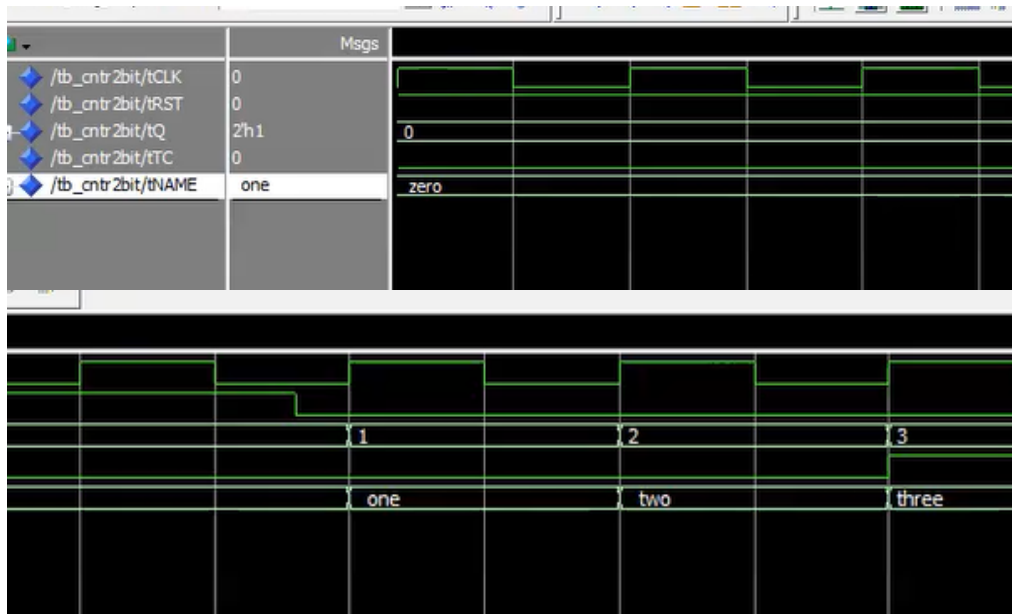
בשורה 19 מופיעה המילה השמורה function, ואחריה את שם הפונקציה, את סוג המשתנה שהיא מקבלת, את סוג המשתנה שהיא מחזירה ואת המילה השמורה is. בשורה 31 מופיעה הפקודה return, שחייבת להופיע בסוף הפונקציה, ומורה מה להחזיר. שאר השורות די דומות לפקודות שראינו עד היום. כאן מופיעה הקריאה לפונקציה:

```

54 T3: tNAME <= TRANSLATOR(tQ);
55

```

את תוצאת הפונקציה אפשר גם להציג בסקופ ב-modelsim, כמו שמופיע כאן (התמונה השנייה היא המשך של ציר הזמן שבתמונה הראשונה):



אפשרויות מתקדמות בכתיבת testbench ב-VHDL גירסה 2008

ראינו בכיתה הדגמה של גישה לאותות פנימיים והדפסתם על המסך, באמצעות פקודות ו-syntax ייעודיים ב-VHDL מהדורת 2008, כמו לדוגמה: כתיבת force ישירות בקוד ה-testbench, שעד גירסת 2008 היתה אפשרויות רק דרך ה-Modelsim. כמו כן, אפשר להציג ערכים של אותות בתור טקסט בחלונות ההודעות של modelsim. אפשר גם לקבוע, שהצגת הטקסט עם ערכי האותות האלה, יתרחש דווקא בפרקי זמן שבהם ערכי האותות השונים הם ערכים שמעניינים אותנו. כך נקבל תדפיס של בדיקות ספציפיות שמעניינות אותנו, ולא נצטרך לחפש את אותם אזורים בתוך ה-waveform.

ב-moodle (ביחידה של מטלה 6) קיימים קובצי ההדגמה ששימשו אותנו בשיעור. בקימפול הקוד שמכיל פקודות אלו יש לכתוב: vcom -2008, כדי ש-modelsim יבין את הפקודות המתקדמות.